

Java Interview Series · Part 12 · Q480–Q529

Collections Framework · Part 1

Hierarchy and choosing a collection, memory and complexity, ArrayList and LinkedList internals, sets, iterators and safe removal, sorting, queues and deques, and the classic puzzles, every example run on a real JDK.

50	49	18	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `CL-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

50	7	49	18
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🗂️	01	Framework Foundations	Q480–Q490	5
📄	02	List Internals & Practice	Q491–Q501	16
🎯	03	Set Internals & Practice	Q502–Q510	27
🔄	04	Iterators & Safe Modification	Q511–Q517	37
🔠	05	Sorting & Searching	Q518–Q522	44
📥	06	Queue & Deque	Q523–Q526	49
🧩	07	Classic Collection Puzzles	Q527–Q529	55

© TechTribers. For personal use only. Do not redistribute.

All questions

🗂️ Framework Foundations

Q480 Draw the Collections Framework hierarchy. Why doesn't Map extend Collection? 5

Q481 What is the difference between Collection, Collections and Arrays? 6

Q482 What are the time complexities of the core operations of each collection? 7

Q483 How do you choose the right collection for a requirement? 8

Q484 Why can't collections hold primitives, and what does boxing cost? 9

Q485 How much memory do the common collections use per element? 10

Q486 Array or ArrayList: when should you still use an array? 11

Q487 How do you convert between arrays, lists, sets and maps? 12

Q488 What are all the ways to iterate a collection, and how do they differ? 13

Q489 What do the most useful Collections utility methods do? 14

Q490 What do the most useful Arrays utility methods do? 15

📃 List Internals & Practice

Q491 How does ArrayList grow, and what do ensureCapacity and trimToSize do? 16

Q492 Why do list.remove(1) behave differently for List<Integer> and List<String>? 18

Q493 What does subList() return, and why can it surprise you? 19

Q494 What is a ListIterator, and what can it do that an Iterator cannot? 20

Q495 Why is LinkedList rarely the right choice, even for frequent insertions? 21

Q496 What are Vector and Stack, and why are they considered legacy? 22

Q497 How do you convert a List to an array correctly? 23

Q498 How do equals() and hashCode() work for the collections themselves? 24

Q499 How do you remove duplicates from a list while keeping the order? 25

Q500 How do you split a list into chunks (batches) of a given size? 25

Q501 How do you remove duplicates from a list of objects by a field (distinct by key)? 26

🎯 Set Internals & Practice

Q502 How is HashSet implemented, and what does add() return? 27

Q503 What happens if you modify an object after adding it to a HashSet? 28

Q504 What happens if you override equals() but not hashCode() (or the reverse)? 30

Q505 How does TreeSet decide whether two elements are equal? 31

Q506 What happens if you add a non-Comparable object to a TreeSet? 31

Q507 What does NavigableSet offer (floor, ceiling, headSet, tailSet…)? 32

Q508 How do you compute the union, intersection and difference of two sets? 33

Q509 Which collections allow null elements, keys or values? 34

Q510 What is a BitSet, and when is it better than Set<Integer> or boolean[]? 36

🔄 Iterators & Safe Modification

Q511 How does ConcurrentModificationException work, and what are the safe ways to remove while iterating? 37

Q512 Why doesn't removing the second-to-last element throw ConcurrentModificationException? 38

Q513 What do removeIf, replaceAll and forEach do on collections? 39

Q514 What is a Spliterator, and how does it relate to Iterator and parallel streams? 40

Q515 How do you make your own class Iterable? (Example: an in-order tree iterator) 41

Q516 How do you build a custom collection quickly with AbstractList? 42

Q517 What is the difference between Iterator and Enumeration? 43

© TechTribers. For personal use only. Do not redistribute.

🔠 Sorting & Searching

Q518 How do you sort by multiple fields with Comparator chaining and null handling? 44

Q519 Why is (a - b) a broken comparator, and what does the Comparator contract require? 45

Q520 How do you sort a Map by values (and by keys)? 46

Q521 Which algorithms do Collections.sort, List.sort, Arrays.sort and Stream.sorted use, and are they stable? 47

Q522 How does binarySearch report that an element was not found? 48

📥 Queue & Deque

Q523 What is the difference between add/offer, remove/poll and element/peek? 49

Q524 How does PriorityQueue work, and why doesn't iteration return sorted order? 50

Q525 ArrayDeque vs LinkedList vs Stack: which should you use as a stack or queue? 52

Q526 Implement a fixed-size ring buffer (circular queue). 53

🧩 Classic Collection Puzzles

Q527 Why does list.get(0) == list.get(1) return true for 100 but false for 1000? 55

Q528 Why does Set<Short>.remove() in a loop never remove anything? 56

Q529 What does Arrays.asList(int[]) return, and why is its size 1? 57

© TechTribers. For personal use only. Do not redistribute.

Section 01

Framework Foundations

The structure of the Collections Framework, how to choose a collection, what each one costs in time and memory, and the utility classes you use every day.

Questions 480–490 · 11 questions



Q480 CL-001 • Basic

Draw the Collections Framework hierarchy. Why doesn't Map extend Collection?

⚡ **Quick answer**

`Iterable` → `Collection` → `List`, `Set`, `Queue` (with `SortedSet` / `NavigableSet` and `Deque` below them). `Map` is a **separate** hierarchy (`Map` → `SortedMap` → `NavigableMap`), because a map holds key-value pairs rather than single elements, so `add(E)` and element-wise `contains` don't fit. Instead it exposes live `Collection` **views**.

Things interviewers add on top of the diagram:

- `Collection` extends `Iterable`, so every collection works in a for-each loop; `Map` does not, you iterate `entrySet()`.
- `Queue` and `Deque` are *interfaces* implemented by `LinkedList`, `ArrayDeque` and `PriorityQueue`.
- Since Java 21, `SequencedCollection`, `SequencedSet` and `SequencedMap` sit in the hierarchy for types with a defined encounter order (`List`, `Deque`, `LinkedHashSet`, `TreeSet`, `LinkedHashMap`, `TreeMap`).
- `Collections`, `Arrays` and `Objects` are utility classes, not part of the interface tree.

Java

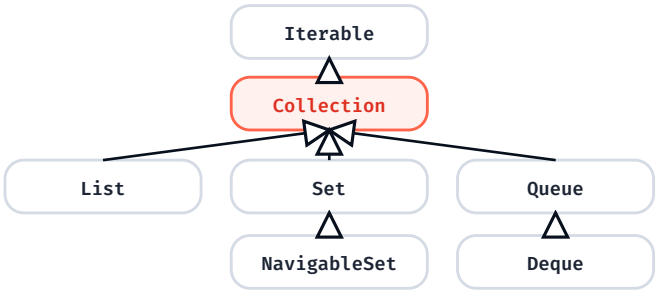
```
Map<String, Integer> m = new LinkedHashMap<>();
m.put("a", 1); m.put("b", 2);
Collection<Integer> values = m.values();                    // views, not copies
Set<String> keys = m.keySet();
Set<Map.Entry<String, Integer>> entries = m.entrySet();
System.out.println(keys + " " + values + " " + entries);
keys.remove("a");                                           // removing through a view removes
from the map
System.out.println(m);
System.out.println((new ArrayList<>() instanceof Collection) + " " + ((Object) m instanceof
Collection));
```

▶ **Output**

```
[a, b] [1, 2] [a=1, b=2]
{b=2}
```

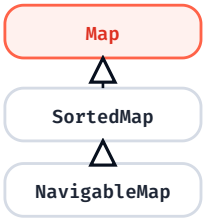

© TechTribers. For personal use only. Do not redistribute.

```
true false
```



Map is NOT a Collection

it stores pairs, so add(E) makes no sense



It exposes Collection VIEWS instead:

`keySet()` · `values()` · `entrySet()`

Typical implementations

`List: ArrayList` · `LinkedList` · `Vector (legacy)`
`Set: HashSet` · `LinkedHashSet` · `TreeSet`
`Queue / Deque: ArrayDeque` · `PriorityQueue` · `LinkedList`
`Map: HashMap` · `LinkedHashMap` · `TreeMap` · `ConcurrentHashMap`

> 💡 **Pro tip**
> A strong one-line answer: "Collection models a group of elements; Map models an association, so it exposes keySet, values and entrySet views instead."

Q481 CL-002 • Basic

What is the difference between Collection, Collections and Arrays?

> ⚡ **Quick answer**
> `Collection` is the **root interface** (a type) for List, Set and Queue. `Collections` is a **utility class** with static methods that operate on collections (`sort` , `reverse` , `unmodifiableList` , `synchronizedList`). `Arrays` is the matching utility class for **arrays** (`sort` , `asList` , `copyOf` , `toString`). Only the first is a type you implement.

Naming convention in the JDK: a plural class name (`Collections` , `Arrays` , `Objects` , `Files` , `Paths`) is a static-method utility class that you never instantiate. A singular name is usually a type. It's the same convention as `Path` (type) and `Paths` (helper).

Java

```
Collection<String> c = new ArrayList<>(List.of("b", "a", "c"));        // Collection: an interface, a type
List<String> list = (List<String>) c;
Collections.sort(list);                                                 // Collections: static helpers for collections
System.out.println(c + " " + Collections.max(c) + " " + Collections.frequency(c, "a"));
String[] arr = {"z", "y"};
Arrays.sort(arr);                                                       // Arrays: static helpers
```


© TechTribers. For personal use only. Do not redistribute.

```
for arrays
System.out.println(Arrays.toString(arr));
```

▶ Output

```
[a, b, c] c 1
[y, z]
```

⚠️ **Interview trap**

You can't call `Collection.sort()` or `list.sort()` for sets: sorting needs an index, which only `List` has. For a `Set`, use a `TreeSet` or copy into a list first.

⏱️ **Q482** CL-003 • Intermediate

What are the time complexities of the core operations of each collection?

⚡ **Quick answer**

`ArrayList` : O(1) get, O(1) amortised append, O(n) middle insert/remove. `LinkedList` : O(n) get, O(1) at ends. `HashSet` / `HashMap` : O(1) average for add, remove, contains. `TreeSet` / `TreeMap` : O(log n). `ArrayDeque` : O(1) at both ends. `PriorityQueue` : O(log n) offer and poll, O(1) peek.

Collection	Add	Remove	Get / contains	Ordered?
`ArrayList`	O(1)* end · O(n) middle	O(n)	get O(1) · contains O(n)	Insertion
`LinkedList`	O(1) ends	O(1) ends · O(n) by index	O(n)	Insertion
`HashSet` / `HashMap`	O(1)	O(1)	O(1)	No
`LinkedHashSet` / `LinkedHashMap`	O(1)	O(1)	O(1)	Insertion / access
`TreeSet` / `TreeMap`	O(log n)	O(log n)	O(log n)	Sorted
`ArrayDeque`	O(1) ends	O(1) ends	O(n)	Insertion
`PriorityQueue`	O(log n)	poll O(log n) · remove(x) O(n)	peek O(1) · contains O(n)	Heap order

* amortised: an occasional append triggers a resize that copies the array. Hash-based O(1) assumes a good `hashCode`; since Java 8 a heavily collided bucket becomes a balanced tree, so the worst case is O(log n), not O(n).

© TechTribers. For personal use only. Do not redistribute.

	get(i) / index	add (end)	insert / remove middle	contains / search
ArrayList	O(1)	O(1)*	O(n)	O(n)
LinkedList	O(n)	O(1)	O(1)†	O(n)
HashSet / HashMap	O(1)	O(1)	O(1)	O(1)
TreeSet / TreeMap	O(log n)	O(log n)	O(log n)	O(log n)
ArrayDeque	n/a	O(1)	ends only	O(n)
PriorityQueue	peek O(1)	O(log n)	poll O(log n)	O(n)

* amortised (occasional resize) † once you already hold the position (iterator/node) HashMap worst case O(log n) since Java 8 (tree bins)

💡 **Pro tip**

Complexity is only half the answer. At small sizes (under a few hundred elements) cache locality often makes `ArrayList` beat everything else, including for inserts in the middle.

🧭 **Q483** CL-004 • Intermediate

How do you choose the right collection for a requirement?

⚡ **Quick answer**

Ask three questions in order: (1) Is it **key → value**, **unique elements**, or a **sequence/queue**? (2) What **ordering** do I need (none, insertion, sorted)? (3) Is it accessed by **many threads**? Default to `ArrayList`, `HashMap` and `HashSet`, and switch only for a concrete reason.

Practical shortcuts:

- Need to look things up by an id → `HashMap<Id, Entity>`.
- Need "unique, in the order I saw them" → `LinkedHashSet`.
- Need "smallest/next above X" → `TreeSet` / `TreeMap` (`NavigableSet` methods).
- Need a stack or a queue → `ArrayDeque` (never `Stack`, rarely `LinkedList`).
- Need "the next highest priority" → `PriorityQueue`.
- Fixed set of enum values → `EnumSet` / `EnumMap`.
- Shared between threads → `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue` (next part), not `Collections.synchronizedX` unless you need its exact semantics.

© TechTribers. For personal use only. Do not redistribute.

```java
List<String> in = List.of("pear", "apple", "fig", "apple", "kiwi");
System.out.println(new HashSet<>(in).size());          // uniqueness only: HashSet, no order guaranteed
System.out.println(new LinkedHashSet<>(in));           // unique + insertion order
System.out.println(new TreeSet<>(in));                 // unique + sorted order
```

▶ Output

```
4
[pear, apple, fig, kiwi]
[apple, fig, kiwi, pear]
```

Key → value Map	Unique elements Set	Sequence / queue List · Queue
sorted by key → TreeMap	sorted → TreeSet	default → ArrayList
insertion/access order → LinkedHashMap	insertion order → LinkedHashSet	stack or queue → ArrayDeque
just fast lookup → HashMap	fast membership → HashSet	by priority → PriorityQueue
many threads → ConcurrentHashMap	enum values → EnumSet	read-mostly, shared → CopyOnWriteArrayList
enum keys → EnumMap	many threads → newKeySet()	producer/consumer → BlockingQueue

⚠️ **Interview trap**

Choosing by Big-O alone is the common mistake. Memory (see the next questions), cache behaviour and iteration order requirements usually matter more than a theoretical O(1) vs O(log n).

📦 **Q484** CL-005 • Intermediate

Why can't collections hold primitives, and what does boxing cost?

⚡ **Quick answer**

Collections are generic and generics work with **reference types** only, so `int` becomes `Integer` through **autoboxing**. Each boxed value outside the small cache is a separate heap object (16 bytes), and unboxing a `null` throws `NullPointerException`. For large numeric data, use arrays, `IntStream`, or a primitive-collection library.

Where the cost shows up: memory (20 bytes per `Integer` in an `ArrayList` vs 4 bytes in an `int[]`, see the next question), allocation and GC pressure in hot loops, and cache misses because boxed values are scattered on the heap.

Alternatives for primitive-heavy code: plain arrays, `IntStream` / `LongStream`, `BitSet` for flags, and third-party primitive collections (Eclipse Collections, fastutil, HPPC). Project Valhalla's value classes (preview from JDK 28) aim to remove most of this overhead for the standard collections too.

© TechTribers. For personal use only. Do not redistribute.

```java
List<Integer> nums = new ArrayList<>();
for (int i = 0; i < 3; i++) nums.add(i);                    // autoboxing: Integer.valueOf(i)
int sum = 0;
for (int n : nums) sum += n;                                 // unboxing on every read
System.out.println(sum);

Map<String, Integer> counts = new HashMap<>();
try { int missing = counts.get("x"); }                       // get() returns null, unboxing it throws
catch (NullPointerException e) { System.out.println("NPE: unboxing a missing map value"); }

int[] prim = nums.stream().mapToInt(Integer::intValue).toArray();  // primitive array: no per-element objects
System.out.println(Arrays.toString(prim) + " " + IntStream.of(prim).sum());
```

▶ Output

```
3
NPE: unboxing a missing map value
[0, 1, 2] 3
```

⚠️ **Interview trap**

`map.getOrDefault(key, 0)` avoids the NPE; `int x = map.get(key)` does not. Also `list.remove(int)` and `list.remove(Object)` behave differently for `List<Integer>` (covered below).

Q485 CL-006 • Advanced

How much memory do the common collections use per element?

⚡ **Quick answer**

Measured on JDK 25 (64-bit, compressed oops) with 1,000,000 boxed `Integer`s: `int[]` **4** bytes/element, `ArrayList` **20**, `ArrayDeque` **21**, `LinkedList` **40**, `HashSet` **57**, `TreeMap` **73**, `HashMap` **73**. The structure around the data often costs more than the data itself.

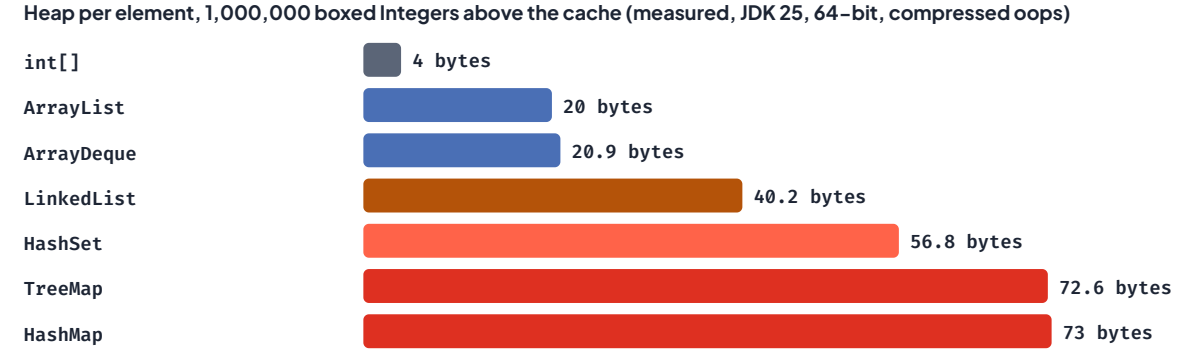
Structure	Bytes per element	Where it goes
`int[]`	4.0	The value itself
`ArrayList<Integer>`	20.0	4-byte reference + 16-byte `Integer`
`ArrayDeque<Integer>`	20.9	Same, plus spare capacity
`LinkedList<Integer>`	40.2	24-byte node (header + 3 references) + `Integer`
`HashSet<Integer>`	56.8	Table slot + 32-byte `HashMap.Node` + `Integer`
`TreeMap<Integer,Integer>`	72.6	40-byte `Entry` + key `Integer` + value `Integer`

© TechTribers. For personal use only. Do not redistribute.

`HashMap<Integer,Integer>`	73.0	Table slot + node + key `Integer` + value `Integer`

Measured here with a small harness (force GC, difference in used heap). Absolute numbers vary with JDK version, heap size and GC; the ratios are stable. With compact object headers (default in JDK 27) every object header shrinks from 12 to 8 bytes, so all the boxed rows get smaller.

```java
long before = usedHeapAfterGc();
List<Integer> list = new ArrayList<>(1_000_000);
for (int i = 0; i < 1_000_000; i++) list.add(i + 1000);        // +1000: above the Integer cache, so every element is its own object
long bytesPerElement = (usedHeapAfterGc() - before) / 1_000_000;
```



ArrayList = 4-byte reference + 16-byte Integer. LinkedList adds a 24-byte node. HashMap adds a 32-byte entry and a second Integer.

💡 Pro tip

Rule of thumb for capacity planning: a `HashMap` entry with small boxed keys and values is roughly 70–100 bytes. 10 million entries is about 0.7–1 GB before your own objects.

VS **Q486** CL-007 • Basic

Array or ArrayList: when should you still use an array?

⚡ Quick answer

Arrays are **fixed-size**, hold **primitives** without boxing, support **multi-dimensional** layouts and are the fastest, most compact choice for numeric data. `ArrayList` is **resizable**, type-safe through generics, and integrates with the collections API and streams. Use arrays for hot numeric loops and fixed layouts, `ArrayList` almost everywhere else.

	Array	`ArrayList`
Size	Fixed	Grows automatically
Elements	Primitives or objects	Objects only (boxing)

© TechTribers. For personal use only. Do not redistribute.

Generics	Reified, covariant, no `new T[]`	Erased, invariant
Utilities	`Arrays.*`	Full Collection API, streams
Memory	Compact	~5× more per `int`

Java

```java
int[] arr = new int[3];                    // fixed size, primitives, length is a field
List<Integer> list = new ArrayList<>();    // resizable, objects only, size() is a method
list.add(1); list.add(2);
System.out.println(arr.length + " " + list.size());

Object[] objs = new String[1];             // arrays are covariant and checked at RUNTIME
(ArrayStoreException)
List<String> strs = new ArrayList<>();
// List<Object> bad = strs;                // generics are invariant: rejected at COMPILE time
int[][] grid = new int[2][3];              // primitives and multi-dimensional data: arrays win
System.out.println(Arrays.deepToString(grid));
```

▶ Output

```
3 2
[[0, 0, 0], [0, 0, 0]]
```

⚠️ **Interview trap**

`array.length` (field), `string.length()` (method) and `collection.size()` (method) are three different spellings, and a classic compile-error question.

Q487 CL-008 • Basic

How do you convert between arrays, lists, sets and maps?

⚡ **Quick answer**

Array → list: `Arrays.asList` (fixed-size view) or `new ArrayList<>(Arrays.asList(a))` (independent copy). List → set: pass it to a set constructor. List → array: `toArray(new T[0])`. List → map: `Collectors.toMap` or a loop with `merge`. Map → list: `new ArrayList<>(map.keySet())` or `values()`. Always decide whether you need a **view** or a **copy**.

`Collectors.toMap` throws `IllegalStateException` on duplicate keys unless you supply a merge function (as above), and `NullPointerException` for null values. `Set.copyOf`, `List.copyOf` and `Map.copyOf` (Java 10) create immutable copies and reject nulls.

Java

```java
String[] arr = {"b", "a", "c", "a"};
List<String> list = Arrays.asList(arr);                    // array -> list (fixed-
size VIEW)
List<String> copy = new ArrayList<>(list);                  // array -> independent,
```


© TechTribers. For personal use only. Do not redistribute.

```
mutable list
Set<String> set = new TreeSet<>(list);                                  // list -> set (sorted, de-
duplicated)
String[] back = list.toArray(new String[0]);                            // list -> array
Map<String, Integer> freq = new TreeMap<>();
for (String s : arr) freq.merge(s, 1, Integer::sum);                    // list -> frequency map
Map<String, Integer> lastIndex = IntStream.range(0, arr.length).boxed()  // array -> value-to-index
map (last one wins)
        .collect(Collectors.toMap(i -> arr[i], i -> i, (x, y) -> y, TreeMap::new));
List<String> keys = new ArrayList<>(freq.keySet());                     // map -> list
System.out.println(copy + " " + set + " " + Arrays.toString(back));
System.out.println(freq + " " + lastIndex + " " + keys);
```

▶ Output

```
[b, a, c, a] [a, b, c] [b, a, c, a]
{a=2, b=1, c=1} {a=3, b=0, c=2} [a, b, c]
```

💡 Pro tip

`Arrays.asList(arr)` writes through to the array and can't change size. `new ArrayList<>(Arrays.asList(arr))` is the usual way to get an independent mutable list.

Q488 CL-009 • Basic

What are all the ways to iterate a collection, and how do they differ?

⚡ Quick answer

Index loop (`List` only), enhanced `for` (uses an `Iterator`), explicit `Iterator` / `ListIterator` (allows `remove` , backwards walk), `forEach` with a lambda (internal iteration), streams (transform while iterating), and since Java 21 `reversed()` for backwards iteration. Pick by need: removal, direction, index access or transformation.

Style	Remove while iterating?	Index?	Notes
Index loop	Yes (careful with indexes)	Yes	Only `List` ; avoid on `LinkedList`
Enhanced `for`	No (CME)	No	Simplest; fail-fast
`Iterator`	Yes, `it.remove()`	No	Works for every collection
`forEach`	No	No	Can't `break` ; lambda can't modify locals
Stream	No (build a new collection)	No	Declarative; parallelisable

Java

```
List<String> names = List.of("Asha", "Ravi", "Meena");
for (int i = 0; i < names.size(); i++) System.out.print(names.get(i) + " ");        // index loop:
great for ArrayList, O(n^2) for LinkedList
System.out.println();
```


© TechTribers. For personal use only. Do not redistribute.

```
for (String n : names) System.out.print(n + " ");                                    // enhanced
for: compiles to an Iterator loop
System.out.println();
for (Iterator<String> it = names.iterator(); it.hasNext(); ) System.out.print(it.next() + " ");   //
explicit Iterator: supports it.remove()
System.out.println();
names.forEach(n -> System.out.print(n + " "));                                       // internal
iteration (Java 8)
System.out.println();
names.stream().map(String::toUpperCase).forEach(n -> System.out.print(n + " "));     // stream:
transform as you go
System.out.println();
for (ListIterator<String> it = names.listIterator(names.size()); it.hasPrevious(); )
System.out.print(it.previous() + " ");    // backwards
System.out.println();
for (String n : names.reversed()) System.out.print(n + " ");                         // Java 21:
reversed view
```

▶ Output

```
Asha Ravi Meena
Asha Ravi Meena
Asha Ravi Meena
Asha Ravi Meena
ASHA RAVI MEENA
Meena Ravi Asha
Meena Ravi Asha
```

⚠️ **Interview trap**

The index loop on a `LinkedList` calls `get(i)` which walks the nodes each time, turning an O(n) loop into O(n²). Use for-each or an iterator for unknown `List` types.

Q489 CL-010 • Basic

What do the most useful Collections utility methods do?

⚡ **Quick answer**

`sort`, `reverse`, `shuffle`, `swap`, `rotate` reorder a list; `max`, `min`, `frequency`, `binarySearch`, `disjoint` query it; `nCopies`, `emptyList`, `singletonList` build tiny lists; `unmodifiableX` and `synchronizedX` return wrapper views.

`Collections.unmodifiableList` is a **view**: changes to the original list show through it (see the unmodifiable-view vs copy question in the Modern Java part). `Collections.synchronizedList` wraps every method in a lock, but iterating still needs manual `synchronized (list)`. `shuffle(list, new Random(seed))` is reproducible, which is handy in tests.

Java

```
List<Integer> nums = new ArrayList<>(List.of(5, 3, 9, 1, 7));
Collections.sort(nums);                                          System.out.println(nums);
```


© TechTribers. For personal use only. Do not redistribute.

```
Collections.reverse(nums);                                    System.out.println(nums);
Collections.shuffle(nums, new Random(42));                    System.out.println(nums.size() + "
" + nums.containsAll(List.of(1, 3, 5, 7, 9)));   // order depends on the seed
Collections.sort(nums); Collections.swap(nums, 0, 4);         System.out.println(nums);
Collections.rotate(nums, 2);                                  System.out.println(nums);
System.out.println(Collections.max(nums) + " " + Collections.min(nums) + " " +
Collections.frequency(nums, 3));
System.out.println(Collections.binarySearch(List.of(1, 3, 5, 7), 5) + " " + Collections.nCopies(3,
"ab"));
System.out.println(Collections.disjoint(List.of(1, 2), List.of(3, 4)) + " " +
Collections.emptyList() + " " + Collections.singletonList(7));
List<Integer> readOnly = Collections.unmodifiableList(nums);      // a read-only VIEW of nums
try { readOnly.add(1); } catch (UnsupportedOperationException e) { System.out.println("read-only
view"); }
```

▶ Output

```
[1, 3, 5, 7, 9]
[9, 7, 5, 3, 1]
5 true
[9, 3, 5, 7, 1]
[7, 1, 9, 3, 5]
9 1 1
2 [ab, ab, ab]
true [] [7]
read-only view
```

💡 **Pro tip**

`Collections.emptyList()` returns a shared immutable singleton; returning it instead of `null` is the standard way to signal "no results".

🧮 **Q490** CL-011 • Basic

What do the most useful Arrays utility methods do?

⚡ **Quick answer**

`sort`, `binarySearch`, `copyOf`, `copyOfRange`, `fill`, `setAll` manipulate arrays; `equals`, `deepEquals`, `hashCode`, `compare`, `mismatch` compare them; `toString` / `deepToString` print them; `asList` and `stream` bridge to collections. Array `equals()` is identity, so always use `Arrays.equals`.

`Arrays.sort(int[])` uses a dual-pivot quicksort; `Arrays.sort(Object[])` and `Arrays.sort(T[], Comparator)` use TimSort (stable). `Arrays.parallelSort` splits large arrays across the common pool. `Arrays.mismatch` (Java 9) returns the first differing index or -1, `Arrays.compare` gives lexicographic order.

Java

```
int[] a = {5, 3, 9, 1};
int[] sorted = a.clone(); Arrays.sort(sorted);
System.out.println(Arrays.toString(sorted) + " " + Arrays.binarySearch(sorted, 5));
System.out.println(Arrays.toString(Arrays.copyOf(a, 6)) + " " +
```


© TechTribers. For personal use only. Do not redistribute.

```
Arrays.toString(Arrays.copyOfRange(a, 1, 3)));
int[] f = new int[4]; Arrays.fill(f, 7); System.out.println(Arrays.toString(f));
System.out.println(Arrays.equals(new int[]{1, 2}, new int[]{1, 2}) + " " + new int[]{1,
2}.equals(new int[]{1, 2}));
System.out.println(Arrays.deepToString(new int[][]{{1, 2}, {3}}) + " " + Arrays.hashCode(new int[]
{1, 2}));
System.out.println(Arrays.mismatch(new int[]{1, 2, 3}, new int[]{1, 2, 4}) + " " +
Arrays.compare(new int[]{1, 2}, new int[]{1, 3}));
Integer[] boxed = {3, 1, 2};
Arrays.sort(boxed, Comparator.reverseOrder());                    // comparators need object arrays
System.out.println(Arrays.toString(boxed) + " " + Arrays.stream(a).sum());
Arrays.setAll(f, i -> i * i);
System.out.println(Arrays.toString(f));
```

▶ Output

```
[1, 3, 5, 9] 2
[5, 3, 9, 1, 0, 0] [3, 9]
[7, 7, 7, 7]
true false
[[1, 2], [3]] 994
2 -1
[3, 2, 1] 18
[0, 1, 4, 9]
```

⚠️ **Interview trap**

`System.out.println(arr)` prints something like `[I@1b6d3586`. Use `Arrays.toString(arr)` (or `deepToString` for nested arrays).

Section 02

List Internals & Practice

How ArrayList and LinkedList really work, the classic remove/subList traps, and the list operations interviewers ask you to code.

Questions 491–501 · 11 questions



Q491 CL-012

• Advanced

How does ArrayList grow, and what do ensureCapacity and trimToSize do?

⚡ **Quick answer**

An `ArrayList` wraps an `Object[]`. `new ArrayList<>()` allocates **nothing** until the first `add`, then capacity 10; when full it grows by about **1.5×** (`old + (old >> 1)`) and copies the array. `ensureCapacity(n)` pre-grows once; `trimToSize()` shrinks the array to the current size.

© TechTribers. For personal use only. Do not redistribute.

Why it matters: every growth allocates a new array and copies all elements, so appending n items one by one costs O(n) amortised but with several full copies. If you know the final size, `new ArrayList<>(n)` avoids all of them. Memory-wise, a list can hold up to ~50% unused slots right after a growth; `trimToSize()` releases them for long-lived lists.

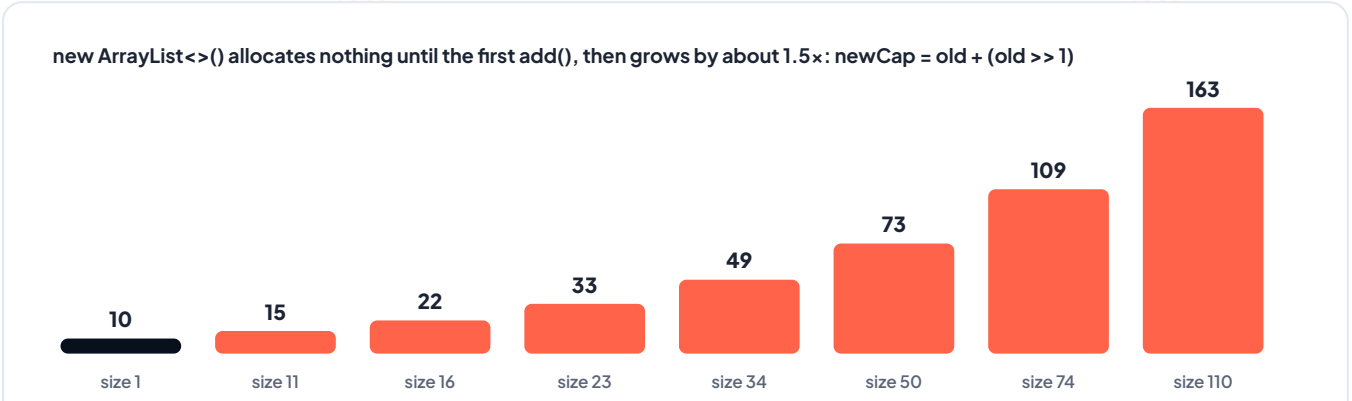
(The sequence above was produced by reading the private `elementData` field via reflection, which is why the program needs `--add-opens java.base/java.util=ALL-UNNAMED`. Don't do that in production code.)

Java

```
static int capacity(ArrayList<?> l) throws Exception {
    Field f = ArrayList.class.getDeclaredField("elementData");      // reflection into JDK internals
(needs --add-opens)
    f.setAccessible(true);
    return ((Object[]) f.get(l)).length;
}
ArrayList<Integer> list = new ArrayList<>();
System.out.println("new ArrayList<>(): capacity " + capacity(list));
int last = capacity(list);
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 120; i++) {
    list.add(i);
    int c = capacity(list);
    if (c != last) { sb.append(" size ").append(i).append("->").append(c).append(";"); last = c; }
}
System.out.println(sb);
ArrayList<Integer> pre = new ArrayList<>(100);
System.out.println("new ArrayList<>(100): " + capacity(pre));
pre.add(1); pre.trimToSize();
System.out.println("after trimToSize with 1 element: " + capacity(pre));
ArrayList<Integer> e = new ArrayList<>(); e.ensureCapacity(500);
System.out.println("ensureCapacity(500): " + capacity(e));
```

▶ Output

```
new ArrayList<>(): capacity 0
 size 1->10; size 11->15; size 16->22; size 23->33; size 34->49; size 50->73; size 74->109; size
110->163;
new ArrayList<>(100): 100
after trimToSize with 1 element: 1
ensureCapacity(500): 500
```



© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`new ArrayList<>(0)` and `new ArrayList<>()` behave differently on the first add (0 → 1 → 2 → 3 → 4 → 6..., vs 0 → 10). Capacity is an implementation detail, but the 1.5× growth rule has been stable for many JDK versions.

🗑️ **Q492** CL-013 • Intermediate

Why do list.remove(1) behave differently for List<Integer> and List<String>?

⚡ **Quick answer**

`List` has two overloads: `remove(int index)` and `remove(Object o)`. With `List<Integer>`, a plain `int` argument selects the **index** version (no boxing needed), so `list.remove(1)` removes the element at position 1, not the value 1. To remove by value, pass an `Integer`: `list.remove(Integer.valueOf(1))`.

This is the overload-resolution rule from the OOP part in action: phase 1 (no boxing) finds `remove(int)` for an `int` argument, so boxing to `Integer` never even gets considered. The same trap exists for `Set<Short>` and `Set<Long>` (a later question).

Java

```
List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 1));
nums.remove(1);                          // remove(int index): removes the element AT index 1 (the 20)
System.out.println(nums);
nums.remove(Integer.valueOf(1));         // remove(Object): removes the first element EQUAL to 1
System.out.println(nums);
List<String> words = new ArrayList<>(List.of("a", "b", "a"));
words.remove("a");                       // only the FIRST occurrence is removed
System.out.println(words);
words.removeIf("a"::equals);             // removes ALL matching elements
System.out.println(words);
```

▶ **Output**

```
[10, 30, 1]
[10, 30]
[b, a]
[b]
```

⚠️ **Interview trap**

`list.remove(i)` inside a loop over indexes also shifts the remaining elements left, so the next element is skipped. Iterate backwards, use an iterator, or call `removeIf`.

© TechTribers. For personal use only. Do not redistribute.

Q493 CL-014 • Intermediate

What does subList() return, and why can it surprise you?

⚡ Quick answer

`subList(from, to)` returns a **view** of that range of the original list, not a copy. Writes through the view change the parent, `view.clear()` deletes the range from the parent, and a **structural change to the parent** (add/remove) makes later use of the view throw `ConcurrentModificationException`. Copy it (`new ArrayList<>(base.subList(...))`) when you need independence.

`subList(...).clear()` is the idiomatic O(n) way to remove a range from an `ArrayList`. Because the view holds a reference to the whole parent, keeping a small `subList` alive also keeps the big parent list in memory.

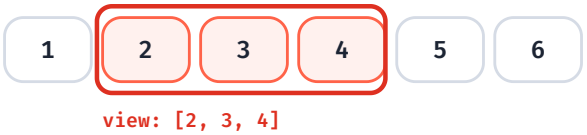
Java

```
List<Integer> base = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
List<Integer> view = base.subList(1, 4);                    // [2, 3, 4] is a VIEW
view.set(0, 99);                                            // writes through to base
System.out.println(base + " " + view);
view.clear();                                               // removes the whole range from the
parent
System.out.println(base);
List<Integer> window = base.subList(0, 2);
base.add(100);                                              // structural change of the parent...
try { window.size(); } catch (ConcurrentModificationException e) {
System.out.println("ConcurrentModificationException on the stale view"); }
List<Integer> safe = new ArrayList<>(base.subList(0, 2));   // copy when you need independence
base.clear();
System.out.println(safe);
```

▶ Output

```
[1, 99, 3, 4, 5, 6] [99, 3, 4]
[1, 5, 6]
ConcurrentModificationException on the stale view
[1, 5]
```

base.subList(1, 4) is a window onto the same storage



view.set(0, 99) writes through to base

view.clear() removes the whole range from base

base.add(...) (structural change) makes the view throw ConcurrentModificationException

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Returning `list.subList(...)` from a method hands callers a live window onto your internal list. Return a copy if the parent can change later.

↔️ **Q494** `CL-015` • Intermediate

What is a ListIterator, and what can it do that an Iterator cannot?

⚡ **Quick answer**
`ListIterator` (for `List` only) extends `Iterator` with **bidirectional** traversal (`hasPrevious`, `previous`), **index** access (`nextIndex`, `previousIndex`) and **modification** during iteration: `set(e)` replaces the last returned element and `add(e)` inserts at the cursor.

The cursor sits *between* elements, so `next()` returns the element it moves over and `previous()` the one on its left. On a `LinkedList`, `add` through a `ListIterator` is O(1) because you already hold the position, while `list.add(index, e)` would first walk to that index.

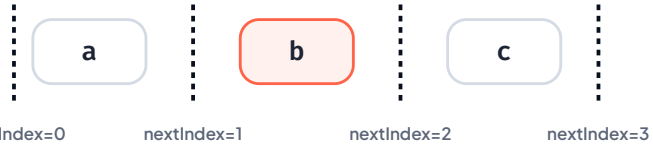
Java

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
ListIterator<String> it = list.listIterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("b")) { it.set("B"); it.add("b2"); }          // replace the element, then insert
after it
    else if (s.equals("c")) it.remove();                       // delete the last returned element
}
System.out.println(list);
while (it.hasPrevious()) System.out.print(it.previous() + " ");   // walk backwards from the end
System.out.println();
```

▶ **Output**

```
[a, B, b2]
b2 B a
```

A ListIterator cursor sits BETWEEN elements



next() moves right and returns the element it passed · previous() moves left
set(e) replaces the element last returned · add(e) inserts at the cursor · remove() deletes the element last returned

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Use `ListIterator` for algorithms that edit a list in place while scanning it (merging, de-duplicating adjacent items, expanding tokens).

🔗 Q495 CL-016 • Advanced

Why is LinkedList rarely the right choice, even for frequent insertions?

> ⚡ **Quick answer**
> Insertion by **index** into a `LinkedList` must first walk to the position, which is O(n) with cache-unfriendly pointer chasing, while `ArrayList` does a fast `System.arraycopy`. In measurements, 50,000 middle inserts took ~1.3–1.6 s on `LinkedList` vs ~60–80 ms on `ArrayList`. `LinkedList` only wins at the **ends** or when you already hold an iterator at the position.

Measured here (JDK 25, 50,000 operations; times vary by machine, ratios are what matter):

Operation	`ArrayList`	`LinkedList`
Insert in the middle by index	~60–80 ms	~1,300–1,600 ms
Insert at the front	~120–140 ms	~0–12 ms
Insert at an iterator position	n/a	~7–10 ms
20,000 random `get(i)`	~1–11 ms	~500–670 ms
Memory per element	20 bytes	40 bytes

"Linked lists are faster for insertion" is true only in the textbook model that ignores finding the position and cache locality. For queues and stacks, `ArrayDeque` beats `LinkedList` in almost every case.

Java

```
LinkedList<String> ll = new LinkedList<>(List.of("a", "b", "c"));
ll.addFirst("start"); ll.addLast("end");                    // O(1) at both ends
ListIterator<String> it = ll.listIterator();
it.next(); it.next();
it.add("inserted");                                         // O(1): the position is already
known
System.out.println(ll + " " + ll.peekFirst() + " " + ll.pollLast());
System.out.println(ll.get(2));                              // get(i) walks node by node from
the nearer end: O(n)
```

▶ **Output**

```
[start, a, inserted, b, c, end] start end
inserted
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Say it as: "I'd use ArrayList by default, ArrayDeque for queues and stacks, and LinkedList only if I need to splice at known iterator positions or need null elements in a Deque."

Q496 CL-017 • Basic

What are Vector and Stack, and why are they considered legacy?

> ⚡ **Quick answer**
> `Vector` (Java 1.0) is a synchronised, growable array; `Stack` extends `Vector` and adds `push` / `pop` / `peek`. They are legacy because **every method is synchronised** (you pay the cost even single-threaded, and compound actions are still not atomic), `Stack` inherits **index-based methods** that break the LIFO abstraction, and `Deque` (`ArrayDeque`) is a better design.

Differences that trip people up: `Stack`'s iteration order is bottom-to-top, `ArrayDeque`'s is top-to-bottom. Replace `Vector` with `ArrayList` (or `CopyOnWriteArrayList` / `Collections.synchronizedList` if you truly need thread safety) and `Stack` with `ArrayDeque`. `Hashtable` and `Enumeration` are in the same legacy family.

Java

```
Stack<Integer> legacy = new Stack<>();                  // extends Vector: synchronized, and exposes get(index)
legacy.push(1); legacy.push(2); legacy.push(3);
System.out.println(legacy.peek() + " " + legacy.get(0) + " " + legacy);    // get(0) is the BOTTOM: a stack shouldn't allow this

Deque<Integer> stack = new ArrayDeque<>();              // the modern choice
stack.push(1); stack.push(2); stack.push(3);
System.out.println(stack.peek() + " " + stack);          // push adds at the FRONT, so iteration is top-first

System.out.println(legacy.iterator().next() + " " + stack.iterator().next());    // opposite iteration orders!
System.out.println(Vector.class.getSuperclass().getSimpleName() + " " +
Stack.class.getSuperclass().getSimpleName());
```

▶ **Output**

```
3 1 [1, 2, 3]
3 [3, 2, 1]
1 3
AbstractList Vector
```

> ⚠️ **Interview trap**
> Synchronised methods don't make compound operations safe: `if (!v.contains(x)) v.add(x)` is still a race. Thread-safety needs the right *abstraction*, not just locked methods.

© TechTribers. For personal use only. Do not redistribute.

Q497 CL-018

• Intermediate

How do you convert a List to an array correctly?

⚡ Quick answer

`list.toArray()` returns `Object[]` (the element type is lost). Pass an array or generator to keep the type: `list.toArray(new String[0])` (classic) or `list.toArray(String[]::new)` (Java 11). A zero-length array is enough; the argument only supplies the runtime type, and a larger array is reused with a `null` terminator.

Old advice said to pre-size the array (`new String[list.size()]`) for speed. Modern JVMs optimise the zero-length form just as well (and it is safer under concurrent modification), so `toArray(new T[0])` or `toArray(T[]::new)` is the recommended style.

Java

```
List<String> list = List.of("a", "b", "c");
Object[] objs = list.toArray();                          // Object[]: the element type is
lost
String[] a1 = list.toArray(new String[0]);               // classic: the argument only
supplies the runtime type
String[] a2 = list.toArray(String[]::new);               // Java 11+: generator function
System.out.println(objs.getClass().getSimpleName() + " " + a1.getClass().getSimpleName() + " " +
a2.length);
try { String[] bad = (String[]) objs; }
catch (ClassCastException e) { System.out.println("ClassCastException: Object[] is not String[]"); }
String[] big = new String[5];
Arrays.fill(big, "x");
String[] filled = list.toArray(big);                     // a large enough array is reused;
the slot after the data becomes null
System.out.println(Arrays.toString(filled) + " " + (filled == big));
```

▶ Output

```
Object[] String[] 3
ClassCastException: Object[] is not String[]
[a, b, c, null, x] true
```

⚠️ Interview trap

Primitive lists have no `toArray(int[])` : convert with `list.stream().mapToInt(Integer::intValue).toArray()` .

© TechTribers. For personal use only. Do not redistribute.

Q498 CL-019 • Intermediate

How do equals() and hashCode() work for the collections themselves?

⚡ Quick answer

Collection equality is defined by **content and interface type**, not by implementation. Two `List`s are equal if they hold equal elements **in the same order**; two `Set`s if they hold the same elements in any order; two `Map`s if they have equal entries. A `List` never equals a `Set`, and an array's `equals()` is identity, so use `Arrays.equals`.

`List.hashCode()` is defined as `31 * h + e.hashCode()` over the elements in order; `Set.hashCode()` is the **sum** of element hashes (so order doesn't matter); `Map.hashCode()` is the sum of entry hashes. These are specified in the interfaces, which is why different implementations interoperate in `equals` and as map keys.

Java

```
List<Integer> arrayList = new ArrayList<>(List.of(1, 2, 3));
List<Integer> linked = new LinkedList<>(List.of(1, 2, 3));
System.out.println(arrayList.equals(linked) + " " + (arrayList.hashCode() == linked.hashCode()) + "
" + arrayList.equals(List.of(1, 2, 3)));   // element-wise, in order
System.out.println(new HashSet<>(List.of(1, 2)).equals(new TreeSet<>(List.of(2, 1))));
// sets: same elements, any order or implementation
System.out.println(List.of(1, 2).equals(Set.of(1, 2)));
// a List never equals a Set
Map<String, Integer> m1 = new HashMap<>(Map.of("a", 1)), m2 = new TreeMap<>(Map.of("a", 1));
System.out.println(m1.equals(m2));
int[] x = {1, 2}, y = {1, 2};
System.out.println(x.equals(y) + " " + Arrays.equals(x, y));
System.out.println(arrayList.hashCode());
// 31 * (31 * (31 * 1 + 1) + 2) + 3
```

▶ Output

```
true true true
true
false
true
false true
30817
```

⚠️ Interview trap

Using a mutable collection as a `HashMap` key or `HashSet` element is dangerous: modifying it changes its `hashCode`, which strands the entry (see the mutated-key question).

© TechTribers. For personal use only. Do not redistribute.

Q499 CL-020

• Basic

How do you remove duplicates from a list while keeping the order?

⚡ Quick answer

Copy it through a `LinkedHashSet` (keeps first occurrences in insertion order, O(n)), use `stream().distinct()`, or loop with a `HashSet` and `add()`'s boolean result. A plain `HashSet` removes duplicates but loses the order.

All of these rely on `equals` / `hashCode` of the element type. For custom objects that don't implement them, you need a key (see the distinct-by-field question) or the objects count as all different.

Java

```
List<String> raw = List.of("pear", "apple", "pear", "fig", "apple");
System.out.println(new ArrayList<>(new LinkedHashSet<>(raw)));           // keeps the first
occurrence, O(n)
System.out.println(raw.stream().distinct().toList());                    // stream version (also
keeps encounter order)
System.out.println(new ArrayList<>(new HashSet<>(raw)).size());          // HashSet removes
duplicates but the ORDER is lost
List<String> loop = new ArrayList<>();
Set<String> seen = new HashSet<>();
for (String s : raw) if (seen.add(s)) loop.add(s);                       // add() returns false
for a duplicate
System.out.println(loop);
```

▶ Output

```
[pear, apple, fig]
[pear, apple, fig]
3
[pear, apple, fig]
```

💡 Pro tip

Interviewers often ask for the O(n²) nested-loop answer first and the O(n) hash-based answer second. State the trade-off: the set version needs O(n) extra memory.

Q500 CL-021

• Intermediate

How do you split a list into chunks (batches) of a given size?

⚡ Quick answer

Use `subList` windows computed from the chunk index: `chunks = ceil(n / size)` and each chunk is `list.subList(i * size, min(n, (i + 1) * size))`. The result holds **views** of the original; copy each chunk if the source may change. `Collectors.partitioningBy` is a different operation (split by a predicate into two groups).

© TechTribers. For personal use only. Do not redistribute.

Typical use: sending 1,000 ids per database `IN (...)` query or 500 records per API batch call. Guava's `Lists.partition` and Apache Commons' `ListUtils.partition` do the same. For streams of unknown length, process with a counter and flush when the buffer reaches the batch size.

Java

```
static <T> List<List<T>> chunks(List<T> list, int size) {
    return IntStream.range(0, (list.size() + size - 1) / size)
            .mapToObj(i -> list.subList(i * size, Math.min(list.size(), (i + 1) * size)))
            .toList();
}
List<Integer> ids = IntStream.rangeClosed(1, 7).boxed().toList();
System.out.println(chunks(ids, 3));
System.out.println(chunks(ids, 10));
System.out.println(chunks(List.<Integer>of(), 3));
System.out.println(ids.stream().collect(Collectors.partitioningBy(i -> i % 2 == 0)));   // a
different problem: split by predicate
```

▶ Output

```
[[1, 2, 3], [4, 5, 6], [7]]
[[1, 2, 3, 4, 5, 6, 7]]
[]
{false=[1, 3, 5, 7], true=[2, 4, 6]}
```

⚠️ Interview trap

`(n + size - 1) / size` is the integer ceiling. Writing `n / size` silently drops the final partial chunk.

Q501 CL-022 • Intermediate

How do you remove duplicates from a list of objects by a field (distinct by key)?

⚡ Quick answer

`stream().distinct()` uses `equals`, so to de-duplicate by one **field** you need a key-based approach: `Collectors.toMap(key, identity, (a, b) -> a, LinkedHashMap::new)` (keeps the first per key), a `filter` with a `seen` set (stateful, keep it sequential), or a `TreeSet` with a key `Comparator`.

Choose by need: option 1 is the cleanest and works in parallel streams; option 2 is the shortest but breaks on parallel streams (shared mutable state); option 3 changes the result order to the comparator's order.

Java

```
record Employee(int id, String name, String dept) { }
List<Employee> staff = List.of(new Employee(1, "Asha", "ENG"), new Employee(2, "Ravi", "ENG"),
        new Employee(3, "Meena", "OPS"), new Employee(4, "Kiran", "OPS"));

// 1) toMap with a merge function keeps the first employee per dept, in encounter order
Collection<Employee> firstPerDept = staff.stream()
        .collect(Collectors.toMap(Employee::dept, e -> e, (a, b) -> a,
LinkedHashMap::new)).values();
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(firstPerDept.stream().map(Employee::name).toList());

// 2) filter with a "seen" set (stateful, so use it on sequential streams only)
Set<String> seen = new HashSet<>();
System.out.println(staff.stream().filter(e -> seen.add(e.dept())).map(Employee::name).toList());

// 3) TreeSet with a key comparator: "equal" means "same dept"
Set<Employee> byDept = new TreeSet<>(Comparator.comparing(Employee::dept));
byDept.addAll(staff);
System.out.println(byDept.stream().map(Employee::name).toList());
```

▶ Output

```
[Asha, Meena]
[Asha, Meena]
[Asha, Meena]
```

⚠️ **Interview trap**

Overriding `equals` / `hashCode` on the class to compare by one field just to enable `distinct()` changes equality **everywhere** that class is used. Prefer a key-based approach for one-off de-duplication.

Section 03

Set Internals & Practice

How HashSet, TreeSet and BitSet work, what breaks when equals/hashCode are wrong, and the set operations interviewers ask for.

Questions 502–510 · 9 questions



Q502 CL-023

• Intermediate

How is HashSet implemented, and what does add() return?

⚡ **Quick answer**

`HashSet` is a thin wrapper around a `HashMap<E, Object>` : each element becomes a **key** and the value is a shared dummy constant (`PRESENT`). `add(e)` is `map.put(e, PRESENT) == null` , so it returns `true` **if the element was new** and `false` if an equal element already existed. Ordering, null support and performance are inherited from `HashMap` .

`LinkedHashSet` is backed by a `LinkedHashMap` and `TreeSet` by a `TreeMap` , the same delegation pattern. Consequences: one `null` element is allowed in `HashSet` / `LinkedHashSet` ; `contains` is O(1) average; and the **memory** cost is that of a map entry (about 57 bytes per boxed `Integer` , see the memory question).

© TechTribers. For personal use only. Do not redistribute.

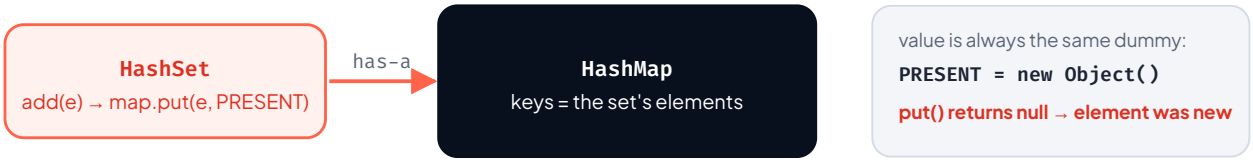
```java
Set<String> set = new HashSet<>();
System.out.println(set.add("java") + " " + set.add("java") + " " + set.add(null) + " " +
set.size());   // add() is false for a duplicate

Map<String, Boolean> map = new HashMap<>();
System.out.println(map.put("java", true) + " " + map.put("java", true));        // put() returns the
previous value: null the first time

Set<String> fromMap = Collections.newSetFromMap(new HashMap<>());               // the same idea,
exposed by the JDK
fromMap.add("x");
System.out.println(fromMap);
System.out.println(new HashSet<>(List.of("a", "b")).getClass().getSuperclass().getSimpleName());
```

▶ Output

```
true false true 2
null true
[x]
AbstractSet
```



add() returns true only when the key was absent. Iteration order, null support and performance all come from HashMap.

💡 **Pro tip**

`Collections.newSetFromMap(new ConcurrentHashMap<>())` (or `ConcurrentHashMap.newKeySet()`) is the standard recipe for a concurrent `Set`.

Q503 CL-024 • Advanced

What happens if you modify an object after adding it to a HashSet?

⚡ **Quick answer**

If the modification changes the object's `hashCode()`, the object is still stored in the bucket for its **old** hash, but lookups now compute the **new** hash and search a different bucket. The result: `contains()` and `remove()` can no longer find it even though it is in the set and iteration still returns it. Use immutable keys, or never mutate fields used in `equals` / `hashCode`.

The same applies to `HashMap` keys and to elements of any hash-based collection. It is a classic source of memory leaks (entries you can never remove) and "ghost" bugs. Defences: make key classes immutable (`final`

© TechTribers. For personal use only. Do not redistribute.

fields, records), or compute `hashCode` only from immutable fields such as an id.

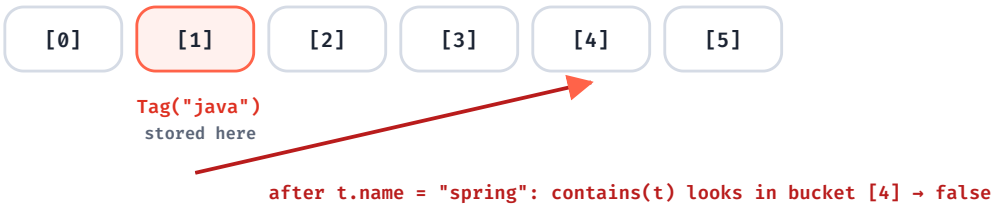
Java

```
class Tag {
    String name;
    Tag(String name) { this.name = name; }
    @Override public boolean equals(Object o) { return o instanceof Tag t && t.name.equals(name); }
    @Override public int hashCode() { return name.hashCode(); }
}
Set<Tag> tags = new HashSet<>();
Tag t = new Tag("java");
tags.add(t);
System.out.println(tags.contains(t));                    // true
t.name = "spring";                                       // hashCode changes while the object sits
in its OLD bucket
System.out.println(tags.contains(t));                    // false: looked up in the NEW bucket
System.out.println(tags.remove(t) + " " + tags.size());  // cannot be removed either
System.out.println(tags.iterator().next().name);         // ...but iteration still finds it
```

▶ Output

```
true
false
false 1
spring
```

Mutating a key after insertion strands it in the wrong bucket



The element is still in the set (iteration finds it) but contains() and remove() can no longer reach it. Use immutable keys.

⚠ **Interview trap**

`String`, `Integer`, `LocalDate` and records are safe keys because they are immutable. A `List` or a mutable POJO as a key is almost always a bug.

© TechTribers. For personal use only. Do not redistribute.

⚖️ Q504 CL-025 • Intermediate

What happens if you override equals() but not hashCode() (or the reverse)?

⚡ Quick answer

The contract says equal objects must have equal hash codes. Override only `equals` and two equal objects usually get **different identity hash codes**, landing in different buckets, so `HashSet.contains` and `HashMap.get` miss them. Override only `hashCode` and objects share buckets but `equals` is still identity, so lookups still fail. `List.contains` only needs `equals`.

The full contract: 1. `equals` is reflexive, symmetric, transitive, consistent, and `equals(null)` is false. 2. If `a.equals(b)` then `a.hashCode() == b.hashCode()`. 3. The reverse is **not** required: unequal objects may share a hash code (a collision), just with a performance cost.

Records generate both methods consistently from their components, which is the easiest way to avoid this bug.

Java

```
class Money1 {                                  // equals only
    final int v; Money1(int v) { this.v = v; }
    @Override public boolean equals(Object o) { return o instanceof Money1 m && m.v == v; }
}
class Money2 {                                  // hashCode only
    final int v; Money2(int v) { this.v = v; }
    @Override public int hashCode() { return v; }
}
Set<Money1> a = new HashSet<>(List.of(new Money1(5)));
System.out.println(a.contains(new Money1(5)));      // false: different identity hash codes put them
in different buckets
Set<Money2> b = new HashSet<>(List.of(new Money2(5)));
System.out.println(b.contains(new Money2(5)));      // false: same bucket, but equals() is still
reference identity
List<Money1> list = List.of(new Money1(5));
System.out.println(list.contains(new Money1(5)));   // true: List.contains only needs equals()
```

▶ Output

```
false
false
true
```

💡 Pro tip

A quick review check: whenever you see `equals` overridden in a PR, look for `hashCode` right below it. IDE generators, Lombok `@EqualsAndHashCode` and records all produce the pair.

© TechTribers. For personal use only. Do not redistribute.

Q505 CL-026

• Advanced

How does TreeSet decide whether two elements are equal?

⚡ Quick answer

`TreeSet` (and `TreeMap`) never call `equals()`. They use `compareTo` (or the supplied `Comparator`): two elements are duplicates when the comparison returns 0**. If the ordering is not *consistent with equals*, the set behaves differently from a `HashSet`: `BigDecimal("1.0")` and `BigDecimal("1.00")` are different to `equals` (scale differs) but equal to `compareTo`.

A comparator inconsistent with `equals` is legal, but you must document it, because the `Set` contract (defined in terms of `equals`) no longer describes the behaviour. Sorting by a single field (`comparing(Employee::age)`) in a `TreeSet` silently drops every other employee with the same age.

Java

```
Set<BigDecimal> hash = new HashSet<>(List.of(new BigDecimal("1.0"), new BigDecimal("1.00")));
Set<BigDecimal> tree = new TreeSet<>(List.of(new BigDecimal("1.0"), new BigDecimal("1.00")));
System.out.println(hash.size() + " " + tree.size());          // equals() sees different scales,
compareTo() does not

Set<String> ci = new TreeSet<>(String.CASE_INSENSITIVE_ORDER); // the comparator defines "duplicate"
ci.addAll(List.of("Java", "JAVA", "java", "Kotlin"));
System.out.println(ci + " " + ci.contains("KOTLIN"));
```

▶ Output

```
2 1
[Java, Kotlin] true
```

⚠️ Interview trap

Using a `TreeSet<Employee>` ordered by salary to "keep employees sorted" loses all employees with a duplicate salary. Add a tie-breaker (`thenComparing(Employee::id)`) or use a sorted `List` instead.

Q506 CL-027

• Intermediate

What happens if you add a non-Comparable object to a TreeSet?

⚡ Quick answer

It throws `ClassCastException` (since Java 7 even for the very first element, because `TreeMap.put` compares the key with itself as a type check). `TreeSet` also rejects `null` with `NullPointerException` when using natural ordering. Fix it by implementing `Comparable` or passing a `Comparator`.

The same rules apply to `TreeMap` keys, `PriorityQueue` elements, `Collections.sort`, `Arrays.sort(Object[])` and `Stream.sorted()` without a comparator: elements must be mutually comparable, or the sort fails at runtime, not at compile time.

© TechTribers. For personal use only. Do not redistribute.

```java
class Plain { }
try { new TreeSet<Plain>().add(new Plain()); }
catch (ClassCastException e) { System.out.println("ClassCastException: not Comparable"); }
try { new TreeSet<String>().add(null); }
catch (NullPointerException e) { System.out.println("NullPointerException: no null elements"); }
TreeSet<Plain> ok = new TreeSet<>(Comparator.comparingInt(Object::hashCode));      // supply a Comparator instead
ok.add(new Plain());
System.out.println(ok.size());
```

▶ Output

```
ClassCastException: not Comparable
NullPointerException: no null elements
1
```

💡 **Pro tip**

The compile-time-friendlier design is to always pass an explicit `Comparator` when sorting domain objects, so the sort order is visible at the call site.

Q507 CL-028 • Intermediate

What does NavigableSet offer (floor, ceiling, headSet, tailSet…)?

⚡ **Quick answer**

`TreeSet` implements `NavigableSet`: `floor(x)` (greatest ≤ x), `lower(x)` (greatest < x), `ceiling(x)` (least ≥ x), `higher(x)` (least > x), `first` / `last`, `pollFirst` / `pollLast`, `headSet` / `tailSet` / `subSet` range **views**, and `descendingSet`. All run in O(log n), which makes `TreeSet` / `TreeMap` ideal for "nearest value" and range queries.

Range methods return **live views** backed by the original set: modifying the view modifies the set, and inserting an element outside the view's range throws `IllegalArgumentException`. Defaults: `headSet(x)` is exclusive, `tailSet(x)` is inclusive, `subSet(a, b)` is `[a, b)`. `TreeMap` offers the same family (`floorKey`, `ceilingEntry`, `subMap`).

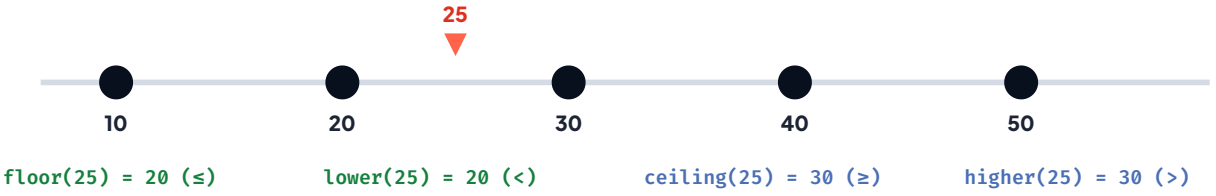
© TechTribers. For personal use only. Do not redistribute.

```java
NavigableSet<Integer> set = new TreeSet<>(List.of(10, 20, 30, 40, 50));
System.out.println(set.floor(25) + " " + set.lower(30) + " " + set.ceiling(25) + " " +
set.higher(30));      // <=, <, >=, >
System.out.println(set.headSet(30) + " " + set.headSet(30, true) + " " + set.tailSet(30) + " " +
set.subSet(20, 40));
System.out.println(set.first() + " " + set.last() + " " + set.pollFirst() + " " +
set.descendingSet());
System.out.println(set.floor(5) + " " + set.ceiling(99));                // null when nothing
qualifies
```

▶ Output

```
20 20 30 40
[10, 20] [10, 20, 30] [30, 40, 50] [20, 30]
10 50 10 [50, 40, 30, 20]
null null
```

TreeSet [10, 20, 30, 40, 50]: navigation relative to a value



At an existing value (30): floor = ceiling = 30, lower = 20, higher = 40 · headSet(x) is exclusive, tailSet(x) inclusive by default

💡 **Pro tip**

Classic uses: finding the nearest available time slot, rate-limit windows, "price just below X", and interval problems (calendar booking with `floor` / `ceiling`).

Q508 CL-029 • Basic

How do you compute the union, intersection and difference of two sets?

⚡ **Quick answer**

Union: `addAll`. Intersection: `retainAll`. Difference: `removeAll`. Symmetric difference: union minus intersection. Operate on a **copy** (`new TreeSet<>(a)`) to keep the originals unchanged. Pass a **set** (not a list) as the argument to `removeAll` / `retainAll` / `containsAll`, otherwise each membership check is O(n).

`ArrayList.removeAll(c)` calls `c.contains(x)` for every element, so passing a `List` makes it O(n·m) and passing a `HashSet` makes it O(n). The same advice applies to `retainAll` and `containsAll`. For two lists of ids, converting one to a `HashSet` first is the standard way to find common or missing elements efficiently.

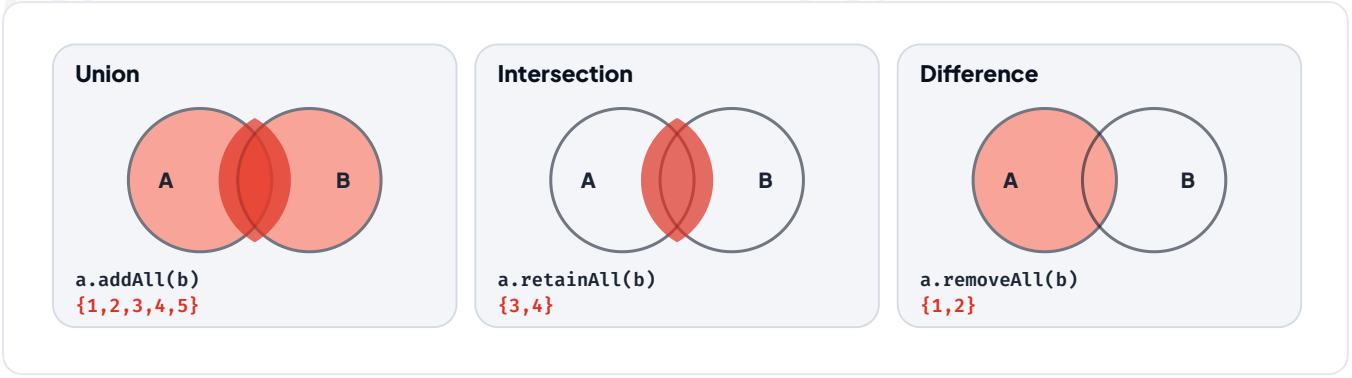
© TechTribers. For personal use only. Do not redistribute.

```java
Set<Integer> a = new TreeSet<>(List.of(1, 2, 3, 4)), b = new TreeSet<>(List.of(3, 4, 5));
Set<Integer> union = new TreeSet<>(a); union.addAll(b);
Set<Integer> inter = new TreeSet<>(a); inter.retainAll(b);
Set<Integer> diff = new TreeSet<>(a); diff.removeAll(b);
Set<Integer> sym = new TreeSet<>(union); sym.removeAll(inter);
System.out.println(union + " " + inter + " " + diff + " " + sym);
System.out.println(a.containsAll(inter) + " " + Collections.disjoint(diff, b));

// Pitfall: removeAll(List) on a big list is O(n * m); convert the argument to a HashSet first
List<Integer> big = new ArrayList<>(IntStream.range(0, 10).boxed().toList());
big.removeAll(new HashSet<>(List.of(2, 4, 6)));
System.out.println(big);
```

▶ Output

```
[1, 2, 3, 4, 5] [3, 4] [1, 2] [1, 2, 5]
true true
[0, 1, 3, 5, 7, 8, 9]
```



⚠️ **Interview trap**

`a.addAll(b)` mutates `a`. If `a` is an immutable set (`Set.of`), it throws `UnsupportedOperationException`. Copy first.

Q509 CL-030 • Intermediate

Which collections allow null elements, keys or values?

⚡ **Quick answer**

`ArrayList`, `LinkedList`, `HashSet`, `LinkedHashSet`, `HashMap` (one null key, null values) and `LinkedHashMap` accept `null`. `TreeSet` / `TreeMap` reject null keys with natural ordering, `Hashtable`, `ConcurrentHashMap`, `ArrayDeque`, `PriorityQueue`, `EnumMap` (keys) and all `List.of` / `Set.of` / `Map.of` collections **reject null**.

Why the differences: sorted structures need to compare elements, so `null` has no ordering; concurrent maps use `null` as the "absent" signal (`get` returning `null` must mean *not present*, which would be ambiguous with a stored null); and `ArrayDeque` / `PriorityQueue` use `null` as the "empty" return value of `poll` / `peek`.

© TechTribers. For personal use only. Do not redistribute.

Java

```
static String tryIt(Runnable r) { try { r.run(); return "ok"; } catch (NullPointerException e) {
return "NPE"; } }
System.out.println("ArrayList.add(null)            " + tryIt(() -> new ArrayList<String>
().add(null)));
System.out.println("LinkedList.add(null)           " + tryIt(() -> new LinkedList<String>
().add(null)));
System.out.println("HashSet.add(null)              " + tryIt(() -> new HashSet<String>
().add(null)));
System.out.println("TreeSet.add(null)              " + tryIt(() -> new TreeSet<String>
().add(null)));
System.out.println("HashMap.put(null, v)           " + tryIt(() -> new HashMap<String, String>
().put(null, "v")));
System.out.println("HashMap.put(k, null)           " + tryIt(() -> new HashMap<String, String>
().put("k", null)));
System.out.println("TreeMap.put(null, v)           " + tryIt(() -> new TreeMap<String, String>
().put(null, "v")));
System.out.println("TreeMap.put(k, null)           " + tryIt(() -> new TreeMap<String, String>
().put("k", null)));
System.out.println("Hashtable.put(k, null)         " + tryIt(() -> new Hashtable<String, String>
().put("k", null)));
System.out.println("ConcurrentHashMap.put(k, null) " + tryIt(() -> new ConcurrentHashMap<String,
String>().put("k", null)));
System.out.println("ArrayDeque.add(null)           " + tryIt(() -> new ArrayDeque<String>
().add(null)));
System.out.println("PriorityQueue.add(null)        " + tryIt(() -> new PriorityQueue<String>
().add(null)));
System.out.println("List.of(null)                  " + tryIt(() -> List.of((String) null)));
```

▶ Output

```
ArrayList.add(null)            ok
LinkedList.add(null)           ok
HashSet.add(null)              ok
TreeSet.add(null)              NPE
HashMap.put(null, v)           ok
HashMap.put(k, null)           ok
TreeMap.put(null, v)           NPE
TreeMap.put(k, null)           ok
Hashtable.put(k, null)         NPE
ConcurrentHashMap.put(k, null) NPE
ArrayDeque.add(null)           NPE
PriorityQueue.add(null)        NPE
List.of(null)                  NPE
```

💡 **Pro tip**

Avoid nulls in collections where you can: return empty collections, use `Optional` for single results, and prefer `Map.getOrDefault` over relying on `null` as a sentinel.

© TechTribers. For personal use only. Do not redistribute.

Q510 CL-031

• Intermediate

What is a BitSet, and when is it better than Set<Integer> or boolean[]?

⚡ **Quick answer**

`BitSet` stores a set of non-negative integers as **one bit per value** in a `long[]`. It is 20–50× more compact than `HashSet<Integer>` and supports fast bitwise operations (`and`, `or`, `xor`, `andNot`, `cardinality`, `nextSetBit`). Use it for dense integer ranges: flags, visited marks, sieves, permissions.

Compared with a `HashSet<Integer>` (about 57 bytes per element), a `BitSet` of values up to 1,000,000 uses roughly 125 KB in total, no matter how many bits are set. It isn't a `Set<Integer>` (it doesn't implement `Collection`), is not thread-safe, and only suits non-negative ints in a bounded range; for sparse or huge values use a `HashSet` or a compressed bitmap such as Roaring.

Java

```
BitSet seen = new BitSet();                                    // one bit per value: 1,000,000
flags need ~125 KB
for (int n : new int[]{3, 5, 8, 5, 3}) seen.set(n);
System.out.println(seen + " " + seen.cardinality() + " " + seen.get(5) + " " + seen.get(6));
BitSet evens = new BitSet();
for (int i = 0; i < 10; i += 2) evens.set(i);
BitSet and = (BitSet) seen.clone(); and.and(evens);
BitSet or = (BitSet) seen.clone(); or.or(evens);
System.out.println(and + " " + or + " " + seen.nextSetBit(4) + " " + seen.nextClearBit(3));

// Sieve of Eratosthenes with a BitSet
BitSet composite = new BitSet(50);
for (int i = 2; i * i < 50; i++) if (!composite.get(i)) for (int j = i * i; j < 50; j += i)
composite.set(j);
System.out.println(IntStream.range(2, 50).filter(i -> !composite.get(i)).boxed().toList());
```

▶ **Output**

```
{3, 5, 8} 3 true false
{8} {0, 2, 3, 4, 5, 6, 8} 5 4
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

⚠️ **Interview trap**

The memory is proportional to the **largest value**, not the number of elements: setting only bit 1,000,000,000 allocates ~125 MB.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Iterators & Safe Modification

How fail-fast iteration works, the safe ways to remove elements, and how to build your own iterable collections.

Questions 511–517 · 7 questions



⚠️ Q511 CL-032

• Intermediate

How does ConcurrentModificationException work, and what are the safe ways to remove while iterating?

⚡ Quick answer

Fail-fast collections keep a `modCount` that increments on every structural change (add/remove). An iterator copies it as `expectedModCount` when created and compares on each `next()`; a mismatch throws `ConcurrentModificationException`. Safe removal: `Iterator.remove()`, `removeIf`, iterating **backwards by index**, or iterating over a **copy**.

Which fix to use: prefer `removeIf` (shortest, and for `ArrayList` it does a single linear pass instead of repeated shifting). Use `Iterator.remove()` when you need to decide per element with extra logic. Iterating a copy costs O(n) extra memory but also protects against *other* code modifying the list.

Fail-fast is **best-effort**: it's meant to expose bugs, not to provide thread safety. Concurrent collections (`ConcurrentHashMap`, `CopyOnWriteArrayList`) use weakly-consistent or snapshot iterators that never throw it.

Java

```java
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
try { for (Integer n : nums) if (n % 2 == 0) nums.remove(n); }
catch (ConcurrentModificationException e) { System.out.println("CME with for-each + remove"); }

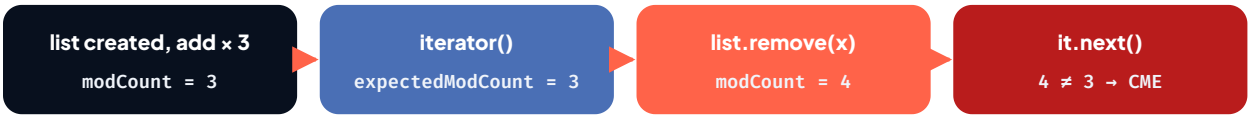
nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
for (Iterator<Integer> it = nums.iterator(); it.hasNext(); ) if (it.next() % 2 == 0) it.remove();
// fix 1: Iterator.remove()
System.out.println(nums);
nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
nums.removeIf(n -> n % 2 == 0);
// fix 2: removeIf
System.out.println(nums);
nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
for (int i = nums.size() - 1; i >= 0; i--) if (nums.get(i) % 2 == 0) nums.remove(i);
// fix 3: backwards by index
System.out.println(nums);
List<Integer> copy = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
for (Integer n : new ArrayList<>(copy)) if (n % 2 == 0) copy.remove(n);
// fix 4: iterate over a copy
System.out.println(copy);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
CME with for-each + remove
[1, 3, 5]
[1, 3, 5]
[1, 3, 5]
[1, 3, 5]
```

Fail-fast: the iterator remembers modCount; any other structural change makes next() throw



it.remove() is safe: it changes the list AND updates expectedModCount in the same step.

Detection is best-effort (a debugging aid), not a guarantee of thread safety.

⚠️ **Interview trap**

You don't need multiple threads to get this exception. The most common cause is a single thread calling `list.remove(...)` inside a for-each loop over the same list.

🎭 **Q512** CL-033 • Advanced

Why doesn't removing the second-to-last element throw ConcurrentModificationException?

⚡ **Quick answer**

The for-each loop calls `hasNext()` before `next()`. `ArrayList`'s `hasNext()` is simply `cursor != size`. After removing the second-to-last element, `size` shrinks by one and now equals `cursor`, so `hasNext()` returns `false` and the loop ends **before** `next()` gets to check the modification count.

Walk through it: after visiting `c` (index 2) the cursor is 3; removing `c` makes `size` = 3. `hasNext()` evaluates `3 != 3` → false, so `d` is silently skipped and no check ever ran. Removing any other element leaves `cursor != size`, `next()` runs `checkForComodification()`, and the exception is thrown.

Java

```
List<String> list = new ArrayList<>(List.of("a", "b", "c", "d"));
for (String s : list) { if (s.equals("c")) list.remove(s); }        // removes the second-to-last
element: NO exception
System.out.println(list);

list = new ArrayList<>(List.of("a", "b", "c", "d"));
try { for (String s : list) { if (s.equals("b")) list.remove(s); } }
catch (ConcurrentModificationException e) { System.out.println("CME when removing a middle
element"); }
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[a, b, d]
CME when removing a middle element
```

⚠️ Interview trap

The fact that it "works" in this one case is a bug hiding as a feature: the last element `d` was never processed. Never rely on it.

Q513 CL-034 • Basic

What do removeIf, replaceAll and forEach do on collections?

⚡ Quick answer

Java 8 added bulk default methods: `Collection.removeIf(predicate)` deletes matching elements and returns whether anything was removed; `List.replaceAll(operator)` replaces each element in place; `Iterable.forEach(consumer)` and `Map.forEach` / `replaceAll` iterate internally. They are shorter than loops and often faster, since `ArrayList` overrides them with one linear pass.

`removeIf` on an `ArrayList` marks matches and compacts the array once (O(n)), while a hand-written loop calling `remove(i)` shifts the tail for each removal (O(n²) in the worst case). Lambdas used here can't modify local variables (they must be effectively final) and `forEach` can't `break`; use a normal loop when you need early exit.

Java

```
List<String> names = new ArrayList<>(List.of("asha", "ravi", "", "meena"));
names.removeIf(String::isEmpty);
names.replaceAll(s -> s.substring(0, 1).toUpperCase() + s.substring(1));
names.forEach(n -> System.out.print(n + " "));
System.out.println();

Map<String, Integer> stock = new TreeMap<>(Map.of("pen", 5, "ink", 0, "pad", 2));
stock.entrySet().removeIf(e -> e.getValue() == 0);              // remove entries through the
entrySet view
stock.replaceAll((k, v) -> v * 10);
stock.forEach((k, v) -> System.out.print(k + "=" + v + " "));
System.out.println();

Collection<Integer> col = new ArrayList<>(List.of(1, 2, 3, 4));
System.out.println(col.removeIf(i -> i > 2) + " " + col.removeIf(i -> i > 9) + " " + col);      //
returns true if anything was removed
```

▶ Output

```
Asha Ravi Meena
pad=20 pen=50
true false [1, 2]
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> For maps, `map.values().removeIf(...)`, `map.keySet().removeIf(...)` and `map.entrySet().removeIf(...)` are the safe in-place removal idioms; the views write through.

Q514 CL-035 • Advanced

What is a Spliterator, and how does it relate to Iterator and parallel streams?

> ⚡ **Quick answer**
> A `Spliterator` (Java 8) is an iterator that can also **split itself**: `trySplit()` hands off roughly half of the remaining elements to a new spliterator so work can run in parallel, `tryAdvance` processes one element, `forEachRemaining` the rest, and `characteristics()` reports properties such as `SIZED`, `ORDERED`, `DISTINCT`, `SORTED`, `IMMUTABLE`. Streams are built on them.

Why it matters: `list.parallelStream()` recursively calls `trySplit()` to divide the work between ForkJoin workers. A collection that splits **evenly and cheaply** (`ArrayList`, arrays, `HashMap` buckets) parallelises well; one that doesn't (`LinkedList`, `Stream.iterate`) doesn't. If you implement your own collection and want good parallel behaviour, override `spliterator()`.

`Collection.spliterator()` default wraps the iterator and can't split well, which is why custom collections that rely on the default get poor parallel performance.

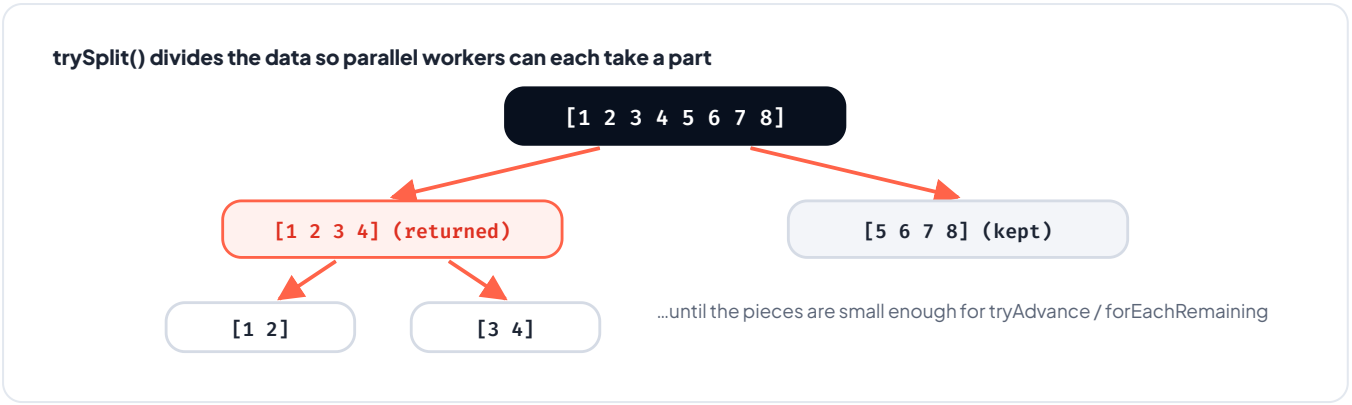
Java

```
List<Integer> list = IntStream.rangeClosed(1, 8).boxed().toList();
Spliterator<Integer> right = list.spliterator();
Spliterator<Integer> left = right.trySplit();                    // splits off the FIRST half;
'right' keeps the rest
left.forEachRemaining(x -> System.out.print(x + " "));
System.out.println();
right.tryAdvance(x -> System.out.println("first of the right half: " + x));     // process exactly
ONE element
right.forEachRemaining(x -> System.out.print(x + " "));
System.out.println();
System.out.println(list.spliterator().hasCharacteristics(Spliterator.SIZED | Spliterator.ORDERED) +
" "
        + new HashSet<>(list).spliterator().hasCharacteristics(Spliterator.ORDERED) + " " +
list.spliterator().estimateSize());
```

▶ **Output**

```
1 2 3 4
first of the right half: 5
6 7 8
true false 8
```


© TechTribers. For personal use only. Do not redistribute.



> 💡 **Pro tip**
> You rarely write a spliterator, but knowing `StreamSupport.stream(iterable.spliterator(), false)` is the standard way to turn any `Iterable` into a `Stream`.

🌳 **Q515** CL-036 • Intermediate

How do you make your own class Iterable? (Example: an in-order tree iterator)

> ⚡ **Quick answer**
> Implement `Iterable<T>` with one method, `iterator()`, returning an `Iterator<T>` that provides `hasNext()` and `next()`. Then the class works in a for-each loop and with `forEach`, and you can create a stream through `spliterator()`. For a tree, keep an explicit stack so the iterator is lazy and uses O(height) memory.

Design points interviewers look for: `iterator()` must return a **fresh** iterator every call (so nested loops work), `next()` must throw `NoSuchElementException` when exhausted, `remove()` defaults to `UnsupportedOperationException`, and the iterator should be lazy rather than copying the whole tree into a list.

Java

```
class Bst implements Iterable<Integer> {
    private record Node(int val, Node left, Node right) { }
    private Node root;
    void add(int v) { root = add(root, v); }
    private Node add(Node n, int v) {
        if (n == null) return new Node(v, null, null);
        return v < n.val() ? new Node(n.val(), add(n.left(), v), n.right()) : new Node(n.val(),
n.left(), add(n.right(), v));
    }
    @Override public Iterator<Integer> iterator() {                // in-order iterator with an
explicit stack
        Deque<Node> stack = new ArrayDeque<>();
        for (Node n = root; n != null; n = n.left()) stack.push(n);
        return new Iterator<>() {
            public boolean hasNext() { return !stack.isEmpty(); }
            public Integer next() {
                if (stack.isEmpty()) throw new NoSuchElementException();
```


© TechTribers. For personal use only. Do not redistribute.

```
                Node n = stack.pop();
                for (Node c = n.right(); c != null; c = c.left()) stack.push(c);
                return n.val();
            }
        };
    }
}
Bst t = new Bst();
for (int v : new int[]{5, 2, 8, 1, 9, 3}) t.add(v);
for (int v : t) System.out.print(v + " ");                    // works in for-each because
Bst is Iterable
System.out.println();
System.out.println(StreamSupport.stream(t.spliterator(),
false).map(String::valueOf).collect(Collectors.joining(",")));
```

▶ Output

```
1 2 3 5 8 9
1,2,3,5,8,9
```

⚠️ **Interview trap**

Returning the same iterator instance from `iterator()` makes a second for-each loop over the object see an already-consumed iterator.

🏗️ **Q516** CL-037 • Intermediate

How do you build a custom collection quickly with AbstractList?

⚡ **Quick answer**

Extend `AbstractList<E>` and implement just `get(int)` and `size()` for a **read-only** list; add `set`, `add(int, E)` and `remove(int)` for a mutable one. You inherit `iterator`, `listIterator`, `contains`, `indexOf`, `equals`, `hashCode`, `subList`, `toString` and more. `AbstractSet`, `AbstractMap` and `AbstractQueue` work the same way.

This is how `Arrays.asList`, `Collections.nCopies` and `List.subList` are built: a small view over some storage with no copying. Here the 5-element list never stores its elements, they are computed on demand, so a `Range(0, 1_000_000_000)` costs a few bytes. Marking it `RandomAccess` tells algorithms such as `Collections.binarySearch` that `get(i)` is cheap.

Java

```
class Range extends AbstractList<Integer> implements RandomAccess {       // only get() and size()
are required
    private final int from, to;
    Range(int from, int to) { this.from = from; this.to = to; }
    @Override public Integer get(int index) { Objects.checkIndex(index, size()); return from +
index; }
    @Override public int size() { return to - from; }
}
List<Integer> r = new Range(10, 15);
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(r + " " + r.contains(12) + " " + r.indexOf(14) + " " + r.subList(1, 3) + " " +
r.stream().mapToInt(i -> i).sum());
try { r.add(1); } catch (UnsupportedOperationException e) { System.out.println("read-only until you
override add/set/remove"); }
System.out.println(new ArrayList<>(r).equals(r) + " " + r.hashCode());
```

▶ Output

```
[10, 11, 12, 13, 14] true 4 [11, 12] 60
read-only until you override add/set/remove
true 38204011
```

💡 **Pro tip**

This is also the right shape for lazy or virtual lists (paged database results, a huge generated sequence) behind the standard `List` interface.

Q517 CL-038 • Basic

What is the difference between Iterator and Enumeration?

⚡ **Quick answer**

`Enumeration` (Java 1.0) has `hasMoreElements()` and `nextElement()` and **cannot remove**; it is used by legacy classes (`Vector` , `Hashtable` , `Properties`). `Iterator` (Java 1.2) has `hasNext()` , `next()` and `remove()` , shorter method names, and is **fail-fast**. Use `Iterator` ; adapt legacy APIs with `Collections.list` and `Collections.enumeration` .

`Enumeration` from `Vector.elements()` is **not** fail-fast (modifying the vector while enumerating gives undefined results), while `Vector.iterator()` is. Other legacy bridge: `Iterator` → `Enumeration` with `Collections.enumeration(collection)` , useful when an old API (such as `Servlet` or `JNDI`) wants an `Enumeration` .

Java

```
Vector<String> v = new Vector<>(List.of("a", "b", "c"));
Enumeration<String> en = v.elements();                        // legacy: hasMoreElements /
nextElement, no remove()
while (en.hasMoreElements()) System.out.print(en.nextElement() + " ");
System.out.println();
Iterator<String> it = v.iterator();                           // modern: fail-fast, supports
remove()
it.next(); it.remove();
System.out.println(v);
System.out.println(Collections.list(v.elements()) + " " + Collections.enumeration(List.of(1,
2)).nextElement());
try { for (String s : v) v.add("x"); } catch (ConcurrentModificationException e) {
System.out.println("Iterator is fail-fast"); }
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
a b c
[b, c]
[b, c] 1
Iterator is fail-fast
```

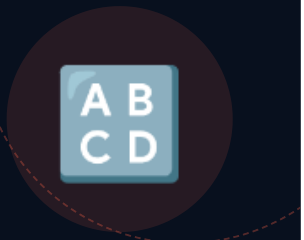
> 💡 **Pro tip**
> Interview shortcut: Enumeration = read-only, legacy, no fail-fast; Iterator = remove supported, fail-fast, the one to use.

Section 05

Sorting & Searching

Comparator chaining, the subtraction trap, sorting maps, stability, and how binary search reports a miss.

Questions 518–522 · 5 questions



Q518 CL-039 • Intermediate

How do you sort by multiple fields with Comparator chaining and null handling?

> ⚡ **Quick answer**
> Build comparators with `Comparator.comparing(keyExtractor)`, then chain tie-breakers with `thenComparing`, flip direction with `reversed()` or `Comparator.reverseOrder()`, and handle nulls with `nullsFirst` / `nullsLast`. Use `comparingInt` / `comparingLong` for primitive keys to avoid boxing.

A subtle point: `Comparator.comparing(Person::age).reversed()` reverses the **whole comparator built so far**. In a chain, write `thenComparing(key, Comparator.reverseOrder())` to reverse only that key. Also, `Comparator.comparing(Person::age)` on a null `Integer` throws `NullPointerException`; wrap the key comparator with `nullsFirst` / `nullsLast`.

Java

```
record Person(String name, Integer age, String city) { }
List<Person> people = new ArrayList<>(List.of(new Person("Ravi", 30, "Pune"), new Person("Asha", 25,
"Pune"), new Person("Meena", 30, "Delhi"),
        new Person("Kiran", null, "Pune"), new Person("Asha", 22, "Mumbai")));
people.sort(Comparator.comparing(Person::city)                                    // 1st key:
city ascending
        .thenComparing(Person::age, Comparator.nullsLast(Comparator.reverseOrder()))    // 2nd key:
age descending, nulls last
        .thenComparing(Person::name));                                            // tie-
breaker: name
people.forEach(p -> System.out.println(p.city() + " " + p.age() + " " + p.name()));
System.out.println(Stream.of("b", null,
```


© TechTribers. For personal use only. Do not redistribute.

```
"a").sorted(Comparator.nullsFirst(Comparator.naturalOrder())).toList());
System.out.println(Stream.of("kiwi", "fig",
"banana").sorted(Comparator.comparingInt(String::length).reversed()).toList());
```

▶ Output

```
Delhi 30 Meena
Mumbai 22 Asha
Pune 30 Ravi
Pune 25 Asha
Pune null Kiran
[null, a, b]
[banana, kiwi, fig]
```

⚠️ **Interview trap**

`list.sort(null)` sorts by natural ordering (elements must be `Comparable`). Passing a lambda like `(a, b) -> a.age() - b.age()` is the next question's trap.

Q519 CL-040 • Advanced

Why is (a - b) a broken comparator, and what does the Comparator contract require?

⚡ **Quick answer**

Subtracting two ints can **overflow** and flip the sign, so `a - b` gives the wrong order for values far apart (`Integer.MIN_VALUE - 1` becomes positive). Use `Integer.compare(a, b)` or `Comparator.comparingInt`. The contract also requires `sgn(compare(a,b)) == -sgn(compare(b,a))`, transitivity, and consistency, otherwise `TimSort` can throw "Comparison method violates its general contract".

The comparator contract (also used by `Comparable`): 1. Antisymmetry: `sgn(compare(a, b)) == -sgn(compare(b, a))`. 2. Transitivity: `compare(a, b) > 0` and `compare(b, c) > 0` imply `compare(a, c) > 0`. 3. Substitutability: `compare(a, b) == 0` implies `compare(a, c)` and `compare(b, c)` have the same sign. 4. Recommended: consistent with `equals` (else document it).

Other classic violations: comparing `double`s with `(int) (a - b)` (truncates small differences to 0), random comparators for shuffling, and comparing by a mutable field that changes during the sort.

Java

```
Comparator<Integer> broken = (a, b) -> a - b;                    // overflows for values far
apart
System.out.println(broken.compare(Integer.MIN_VALUE, 1) + " " + broken.compare(1,
Integer.MIN_VALUE));    // wrong signs!
Comparator<Integer> correct = Integer::compare;
System.out.println(correct.compare(Integer.MIN_VALUE, 1) + " " + correct.compare(1,
Integer.MIN_VALUE));
List<Integer> data = new ArrayList<>(List.of(1, Integer.MAX_VALUE, -5, Integer.MIN_VALUE));
data.sort(broken);
System.out.println(data);                                         // not sorted
```


© TechTribers. For personal use only. Do not redistribute.

```
data.sort(correct);
System.out.println(data);
```

▶ Output

```
2147483647 -2147483647
-1 1
[1, 2147483647, -2147483648, -5]
[-2147483648, -5, 1, 2147483647]
```

⚠️ Interview trap

The overflow bug passes every test that uses small numbers and fails only on production data with large values or timestamps. `Long.compare`, `Double.compare` and `Integer.compare` exist for exactly this reason.

🗺️ **Q520** CL-041 • Intermediate

How do you sort a Map by values (and by keys)?

⚡ Quick answer

A `HashMap` has no order, so sorting means producing a new ordered structure. By **key**: `new TreeMap<>(map)`. By **value**: stream the `entrySet()`, sort with `Map.Entry.comparingByValue()` (plus a key tie-breaker for deterministic ties) and collect into a *`LinkedHashMap`* so the order is preserved.

Collecting into a plain `HashMap` / `toMap()` after sorting **loses the order** again; `LinkedHashMap::new` as the map supplier is the essential detail. For repeated access by rank, keep a sorted `List<Entry>`; for a continuously updated leaderboard use a `TreeSet` of (score, id) pairs or a `PriorityQueue`.

Java

```
Map<String, Integer> scores = new HashMap<>(Map.of("asha", 82, "ravi", 91, "meena", 82, "kiran",
70));
// by value descending, then key ascending (the tie-breaker makes the order deterministic)
LinkedHashMap<String, Integer> byValue = scores.entrySet().stream()
        .sorted(Map.Entry.<String,
Integer>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey()))
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue, (a, b) -> a,
LinkedHashMap::new));
System.out.println(byValue);
System.out.println(new TreeMap<>(scores));                          // by key: TreeMap
List<Map.Entry<String, Integer>> entries = new ArrayList<>(scores.entrySet());
entries.sort(Map.Entry.comparingByValue());
System.out.println(entries.get(0) + " " + entries.get(entries.size() - 1));
```

▶ Output

```
{ravi=91, asha=82, meena=82, kiran=70}
{asha=82, kiran=70, meena=82, ravi=91}
kiran=70 ravi=91
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Top-N by value without sorting everything: stream the entries through a size-N `PriorityQueue` (min-heap) in O(n log N).

⚙️ Q521 CL-042 • Advanced

Which algorithms do Collections.sort, List.sort, Arrays.sort and Stream.sorted use, and are they stable?

> ⚡ **Quick answer**
> Sorting **objects** (`Collections.sort` , `List.sort` , `Arrays.sort(Object[])` , `Stream.sorted`) uses **TimSort**, a stable merge/insertion hybrid that is O(n log n) worst case and ~O(n) on already-sorted data. Sorting **primitives** (`Arrays.sort(int[])`) uses a dual-pivot quicksort, which isn't stable (irrelevant for primitives). `Collections.sort(list)` just calls `list.sort(null)` .

Why stability matters: it lets you sort in passes. Sort by name, then (stably) by department, and you get department order with names sorted inside each department. `Comparator.thenComparing` does the same in one pass.

`List.sort` copies the elements to an array, sorts it with `Arrays.sort` , and writes it back (`ArrayList` overrides it to sort its internal array directly). `LinkedList` therefore pays for the copy. `Arrays.parallelSort` uses a parallel merge sort for large arrays and is also stable for objects.

Java

```
record Emp(String name, int age) { }
List<Emp> list = new ArrayList<>(List.of(new Emp("Asha", 25), new Emp("Ravi", 30), new Emp("Meena",
25), new Emp("Kiran", 30), new Emp("Dev", 25)));
list.sort(Comparator.comparingInt(Emp::age));                          // object sort = TimSort:
STABLE
System.out.println(list.stream().map(Emp::name).toList());             // equal ages keep their
original relative order

Emp[] arr = list.toArray(new Emp[0]);
Arrays.sort(arr, Comparator.comparingInt(Emp::age).reversed());
System.out.println(Arrays.stream(arr).map(Emp::name).toList());
System.out.println(list.stream().sorted(Comparator.comparingInt(Emp::age)).map(Emp::name).toList());
// stable for ordered streams
int[] prim = {5, 3, 9, 1, 7};
Arrays.sort(prim);                                                     // primitives: dual-pivot
quicksort
System.out.println(Arrays.toString(prim));
```

▶ **Output**

```
[Asha, Meena, Dev, Ravi, Kiran]
[Ravi, Kiran, Asha, Meena, Dev]
[Asha, Meena, Dev, Ravi, Kiran]
[1, 3, 5, 7, 9]
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

Sorting with a comparator that violates the contract can throw `IllegalArgumentException: Comparison method violates its general contract!` in TimSort, but only sometimes (it needs enough elements to trigger the merge logic), which makes the bug intermittent.

🔍 **Q522** CL-043 • Intermediate

How does binarySearch report that an element was not found?

⚡ **Quick answer**

`Arrays.binarySearch` and `Collections.binarySearch` return the element's **index** if found, otherwise `-(insertionPoint) - 1`, where `insertionPoint` is the index where the element would be inserted to keep the array sorted. The result is always negative for a miss, so test `< 0`. The input **must be sorted** (same ordering), or the result is undefined.

The encoding exists so one return value carries both facts (found/not found, and where to insert), which makes "insert if absent into a sorted array" a one-liner: `int i = Arrays.binarySearch(a, x); if (i < 0) insertAt(-i - 1, x);`. On a `LinkedList`, `Collections.binarySearch` falls back to an iterator-based search because the list isn't `RandomAccess`.

Java

```
int[] sorted = {10, 20, 30, 40};
System.out.println(Arrays.binarySearch(sorted, 30) + " " + Arrays.binarySearch(sorted, 25) + " " +
Arrays.binarySearch(sorted, 5) + " " + Arrays.binarySearch(sorted, 99));
int notFound = Arrays.binarySearch(sorted, 25);
System.out.println("insertion point = " + (-notFound - 1));          // -(insertionPoint) - 1  =>
insertionPoint = -result - 1
List<String> names = List.of("asha", "kiran", "ravi");
System.out.println(Collections.binarySearch(names, "meena") + " " + Collections.binarySearch(names,
"zed"));
int[] unsorted = {30, 10, 40, 20};
System.out.println(Arrays.binarySearch(unsorted, 20));              // 20 IS present, but
unsorted input gives a wrong answer silently
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
2 -3 -1 -5
insertion point = 2
-3 -4
-3
```

⚠️ **Interview trap**

With duplicate values, `binarySearch` may return **any** matching index, not the first or last. For lower/upper bounds use `TreeSet` / `TreeMap` (`floor` , `ceiling`) or a custom loop.

Section 06

Queue & Deque

The two method families of Queue, how PriorityQueue really behaves, which structure to use as a stack, and a hand-built ring buffer.

Questions 523–526 · 4 questions



Q523 CL-044 • Basic

What is the difference between add/offer, remove/poll and element/peek?

⚡ **Quick answer**

`Queue` offers two forms of each operation: one that **throws** an exception on failure (`add` , `remove` , `element`) and one that returns a **special value** (`offer` → `false` , `poll` → `null` , `peek` → `null`). Use the throwing forms when failure is a bug, and the value-returning forms when it is normal (full queue, empty queue).

Operation	Throws exception	Returns special value
Insert	`add(e)`	`offer(e)` → `false`
Remove head	`remove()`	`poll()` → `null`
Examine head	`element()`	`peek()` → `null`

For unbounded queues (`LinkedList` , `ArrayDeque` , `PriorityQueue`) `add` and `offer` behave the same, because insertion never fails for capacity reasons. `BlockingQueue` adds two more forms: `put` / `take` (block) and `offer` / `poll` with a timeout.

© TechTribers. For personal use only. Do not redistribute.

```java
Queue<String> bounded = new ArrayBlockingQueue<>(2);                // a bounded queue makes the difference visible
System.out.println(bounded.add("a") + " " + bounded.offer("b") + " " + bounded.offer("c"));      // offer returns false when full
try { bounded.add("c"); } catch (IllegalStateException e) { System.out.println("add throws: " + e.getMessage()); }
System.out.println(bounded.peek() + " " + bounded.poll() + " " + bounded.remove() + " " + bounded.poll() + " " + bounded.peek());   // null when empty
try { bounded.remove(); } catch (NoSuchElementException e) { System.out.println("remove throws on empty"); }
try { bounded.element(); } catch (NoSuchElementException e) { System.out.println("element throws on empty"); }
```

▶ Output

```
true true false
add throws: Queue full
a a b null null
remove throws on empty
element throws on empty
```

💡 **Pro tip**

Prefer `offer` / `poll` / `peek` in loops like `while ((x = queue.poll()) != null)`; it avoids a separate `isEmpty()` check and works with concurrent queues.

Q524 CL-045 • Intermediate

How does PriorityQueue work, and why doesn't iteration return sorted order?

⚡ **Quick answer**

`PriorityQueue` is a **binary min-heap** in an array: the smallest element (by natural order or comparator) is always at index 0, giving O(1) `peek`, O(log n) `offer` / `poll`. The heap only guarantees the **parent ≤ children** property, so `toString()` and iteration walk the *array*, not sorted order. Only repeated `poll()` returns elements in order.

Facts to remember:

- Not thread-safe (use `PriorityBlockingQueue`); no `null` elements; elements must be `Comparable` or you must pass a `Comparator`.
- `remove(Object)` and `contains` are O(n); building from a collection with the constructor is O(n) (heapify), cheaper than n separate `offer`s.
- Ties between equal priorities are **not** FIFO. For stable tie-breaking add a sequence number to the comparator.

The "top K" pattern (a min-heap of size K) runs in O(n log K) and O(K) memory: better than sorting everything when K is small.

© TechTribers. For personal use only. Do not redistribute.

Java

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int n : new int[]{7, 2, 9, 1, 5, 3}) pq.offer(n);
System.out.println(pq);                                  // heap array order, NOT sorted
System.out.println(pq.peek());
StringBuilder sb = new StringBuilder();
while (!pq.isEmpty()) sb.append(pq.poll()).append(' ');  // poll() returns elements in sorted order, O(log n) each
System.out.println(sb);

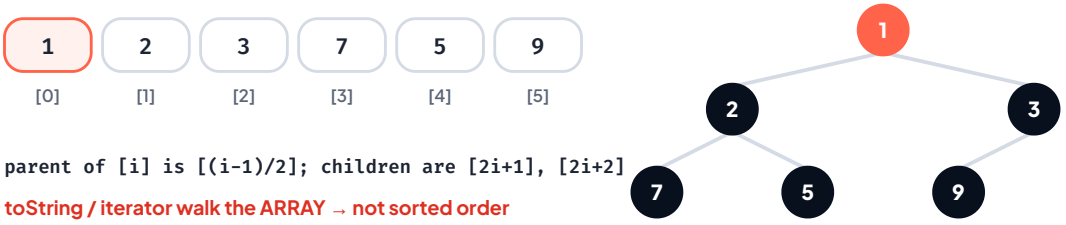
PriorityQueue<String> longestFirst = new PriorityQueue<>
(Comparator.comparing(String::length).reversed());
longestFirst.addAll(List.of("kiwi", "fig", "banana", "plum"));
System.out.println(longestFirst.poll() + " " + longestFirst.poll());

PriorityQueue<Integer> topK = new PriorityQueue<>();      // keep the K largest values with a size-K min-heap
for (int n : new int[]{4, 9, 1, 7, 3, 8}) { topK.offer(n); if (topK.size() > 3) topK.poll(); }
System.out.println(new TreeSet<>(topK).descendingSet());
```

▶ Output

```
[1, 2, 3, 7, 5, 9]
1
1 2 3 5 7 9
banana plum
[9, 8, 7]
```

PriorityQueue is a binary min-heap stored in an array



`parent of [i] is [(i-1)/2]; children are [2i+1], [2i+2]`

toString / iterator walk the ARRAY → not sorted order

poll() removes the root and sifts down → ascending order

⚠️ **Interview trap**

Printing a `PriorityQueue` to check that your comparator works is misleading. Only repeated `poll()` shows the real ordering.

© TechTribers. For personal use only. Do not redistribute.

🔁 Q525 CL-046 • Intermediate

ArrayDeque vs LinkedList vs Stack: which should you use as a stack or queue?

⚡ Quick answer

Use `ArrayDeque`. It's a resizable circular array: O(1) at both ends, no per-element node allocation, no synchronisation, better cache behaviour. `LinkedList` is slower and uses twice the memory (allowed nulls are its only advantage), and `Stack` is a legacy synchronised `Vector`. `push` / `pop` / `peek` work on the front**; `offer` / `poll` add at the back and take from the front.

	`ArrayDeque`	`LinkedList`	`Stack`
Structure	Circular array	Doubly-linked nodes	`Vector` (array)
Ends O(1)	Yes	Yes	Top only
Null elements	No	Yes	Yes
Memory/element	~21 bytes	~40 bytes	~20 bytes
Thread-safe	No	No	Synchronised (slowly)
Also a `List`	No	Yes	Yes

Java

```
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(1); dq.offerLast(2); dq.push(0);              // push = addFirst
System.out.println(dq + " " + dq.peekFirst() + " " + dq.peekLast());
System.out.println(dq.pop() + " " + dq.pollLast() + " " + dq);
try { dq.add(null); } catch (NullPointerException e) { System.out.println("ArrayDeque rejects
null"); }
Deque<Integer> linked = new LinkedList<>();
linked.add(null);                                           // LinkedList allows null (and is
slower: a node per element)
System.out.println(linked.size());

Deque<Integer> stack = new ArrayDeque<>();                  // LIFO
Queue<Integer> queue = new ArrayDeque<>();                  // FIFO
for (int i = 1; i <= 3; i++) { stack.push(i); queue.offer(i); }
System.out.println(stack.pop() + " " + queue.poll());
Iterator<Integer> desc = new ArrayDeque<>(List.of(1, 2, 3)).descendingIterator();
System.out.println(desc.next());
```

▶ Output

```
[0, 1, 2] 0 2
0 2 [1]
ArrayDeque rejects null
1
3 1
3
```


© TechTribers. For personal use only. Do not redistribute.

Deque = double-ended queue: every operation exists at both ends

FIRST end (head)
`addFirst` · `offerFirst` · `push`
`pollFirst` · `removeFirst` · `pop`
`peekFirst` · `getFirst`



LAST end (tail)
`addLast` · `offerLast` · `add`
`pollLast` · `removeLast`
`peekLast` · `getLast`

Stack (LIFO): push / pop / peek at the FIRST end · Queue (FIFO): offer at LAST end, poll from FIRST end

ArrayDeque: resizable circular array, no nulls, no capacity limit, faster than LinkedList and Stack.

> 💡 **Pro tip**
> For a thread-safe queue use `ConcurrentLinkedDeque` or a `BlockingQueue`; for a thread-safe stack, `ConcurrentLinkedDeque` with `push` / `pop`.

🔄 **Q526** CL-047 • Intermediate

Implement a fixed-size ring buffer (circular queue).

> ⚡ **Quick answer**
> Keep an array, a `head` index and a `size`. Offer writes at `(head + size) % capacity`; poll reads at `head`, advances it `(head + 1) % capacity` and decrements `size`. The array is reused in a circle, so nothing shifts. The queue is full when `size == capacity` (offer returns `false`) and empty when `size == 0`.

Keeping `size` separately (instead of two indexes `head` and `tail`) removes the classic ambiguity that `head == tail` means both empty and full. Variants: **overwrite oldest** on full (logging, metrics windows), a power-of-two capacity so `% n` becomes a cheap bit mask (`& (n - 1)`), and a lock-free single-producer/single-consumer version (as in the LMAX Disruptor).

Java

```
class RingBuffer<T> {
    private final Object[] data;
    private int head, size;
    RingBuffer(int capacity) { data = new Object[capacity]; }
    boolean offer(T item) {
        if (size == data.length) return false;                    // full
        data[(head + size) % data.length] = item;
        size++;
        return true;
    }
    @SuppressWarnings("unchecked")
    T poll() {
        if (size == 0) return null;                                // empty
        T item = (T) data[head];
        data[head] = null;                                         // don't hold on to the object
        head = (head + 1) % data.length;
        size--;
        return item;
```

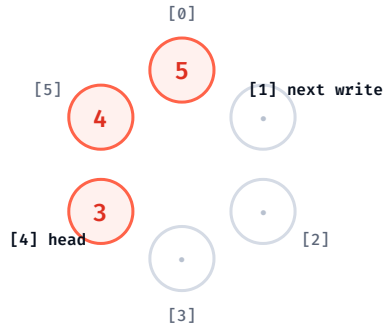

© TechTribers. For personal use only. Do not redistribute.

```
    }
    int size() { return size; }
    @Override public String toString() {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < size; i++) sb.append(i > 0 ? ", " : "").append(data[(head + i) %
data.length]);
        return sb.append("]").toString();
    }
}
RingBuffer<Integer> rb = new RingBuffer<>(3);
System.out.println(rb.offer(1) + " " + rb.offer(2) + " " + rb.offer(3) + " " + rb.offer(4));      //
full: rejects 4
System.out.println(rb.poll() + " " + rb.poll() + " " + rb);
rb.offer(5); rb.offer(6);                                                                          //
wraps around the end of the array
System.out.println(rb + " size=" + rb.size());
```

▶ Output

```
true true true false
1 2 [3]
[3, 5, 6] size=3
```

Ring buffer: head and tail wrap around a fixed array, no shifting needed



capacity 6 · head = 4 · size = 3

elements sit at indexes 4, 5, 0 (wrapped around)

`offer: data[(head + size) % cap] = x`
`poll: x = data[head]; head = (head + 1) % cap`

full when size == capacity → offer returns false

💡 **Pro tip**

`ArrayDeque` is itself a growing ring buffer. Implement the fixed-size version when the interviewer wants bounded memory and a drop/overwrite policy.

© TechTribers. For personal use only. Do not redistribute.

Section 07

Classic Collection Puzzles

The short "what does this print?" questions that reveal whether you understand boxing, overload resolution and array views.

Questions 527–529 · 3 questions



Q527 CL-048

• Intermediate

Why does list.get(0) == list.get(1) return true for 100 but false for 1000?

⚡ Quick answer

`==` on two `Integer` objects compares **references**. Autoboxing calls `Integer.valueOf`, which returns **cached shared objects** for values from -128 to 127 and a **new object** for anything else. So `100 == 100` happens to be the same cached object, while two boxes of `1000` are different objects. Compare boxed values with `equals()`.

The cache range's upper bound is configurable (`-XX:AutoBoxCacheMax=N`), so code that "works" with `==` can break or change with JVM flags. `Long`, `Short`, `Byte` and `Character` have similar caches; `Double` and `Float` don't cache at all. Collections themselves are safe: `HashMap`, `contains`, `indexOf` use `equals`.

Java

```
List<Integer> small = List.of(100, 100), large = List.of(1000, 1000);
System.out.println((small.get(0) == small.get(1)) + " " + (large.get(0) == large.get(1)));
// == compares references
System.out.println(large.get(0).equals(large.get(1)) + " " + (large.get(0).intValue() ==
large.get(1)));   // equals() or unboxing compares values
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d));
Map<Integer, String> map = new HashMap<>();
map.put(1000, "x");
System.out.println(map.get(1000) + " " + map.containsKey(1000));
// fine: lookups use equals()
List<Integer> ids = new ArrayList<>(List.of(1000, 1000));
System.out.println(ids.get(0) == ids.get(1));
// false: two different boxes
```

▶ Output

```
true false
true true
true false
x true
false
```


© TechTribers. For personal use only. Do not redistribute.

Integer.valueOf caches -128 … 127: == works there by accident, and fails outside

< -128	-128 … 127	> 127
new Integer each time	shared cached objects	new Integer each time

`Integer a = 127, b = 127; a == b → true Integer c = 128, d = 128; c == d → false`

Always compare boxed values with equals() (or unbox with intValue()). Collections do this for you, == on elements does not.

⚠️ **Interview trap**

The bug appears in business code like `if (order.getQuantity() == other.getQuantity())` on `Integer` fields, working in tests with small numbers and failing in production.

Q528 CL-049 • Advanced

Why does Set<Short>.remove() in a loop never remove anything?

⚡ **Quick answer**

`i - 1` is an `int` expression (arithmetic on `short` promotes to `int`), so `set.remove(i - 1)` autoboxes to `Integer`. The set contains `Short` objects, and `Integer.valueOf(0).equals(Short.valueOf((short) 0))` is false**, so nothing is removed. `Set.remove(Object)` accepts any object, so the compiler doesn't warn.

This is from *Java Puzzlers*. The root cause: `remove(Object)`, `contains(Object)` and `get(Object)` on collections take `Object`, not `E`, so wrong-typed arguments compile fine and simply never match. `Map.get(Object)` and `Map.containsKey(Object)` have the same trap (`Map<Long, V>` with `map.get(1)` looking up an `Integer`).

Java

```
Set<Short> set = new HashSet<>();
for (short i = 0; i < 100; i++) { set.add(i); set.remove(i - 1); }          // i - 1 is an int, so
it is boxed to Integer, not Short
System.out.println(set.size());                                              // 100, not 1

Set<Short> fixed = new HashSet<>();
for (short i = 0; i < 100; i++) { fixed.add(i); fixed.remove((short) (i - 1)); }
System.out.println(fixed.size());

List<Short> list = new ArrayList<>(List.of((short) 1, (short) 2));
System.out.println(list.contains(1) + " " + list.contains((short) 1) + " " + list.indexOf(2));
```

▶ **Output**

```
100
1
false true -1
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Static analysers flag it (IntelliJ "suspicious call to Collection.remove"). In `Map<Long, String>`, write `map.get(1L)`, never `map.get(1)`.

📦 **Q529** `CL-050` • Intermediate

What does Arrays.asList(int[]) return, and why is its size 1?

> ⚡ **Quick answer**
> `Arrays.asList(T... a)` is generic, and generics can't bind to primitive types, so an `int[]` is treated as **one object** of type `int[]`: `T` becomes `int[]` and you get a `List<int[]>` of size 1. With an `Integer[]` the elements are spread as expected. `List.of(int[])` has the same trap. Use `Arrays.stream(prim).boxed().toList()` for primitives.

Two more `Arrays.asList` facts, both shown above: the list is a **view** (writes go through to the array in both directions) and it has a **fixed size**: `set` works, but `add` and `remove` throw `UnsupportedOperationException`. It's the same pattern as `AbstractList`: a small adapter over existing storage.

Java

```
int[] prim = {1, 2, 3};
List<int[]> wrong = Arrays.asList(prim);                    // T is inferred as int[]: a list with
ONE element
System.out.println(wrong.size() + " " + wrong.get(0).length);
Integer[] boxed = {1, 2, 3};
List<Integer> right = Arrays.asList(boxed);
System.out.println(right.size());
List<Integer> fromStream = Arrays.stream(prim).boxed().toList();      // correct for primitive
arrays
System.out.println(fromStream + " " + List.of(prim).size());          // List.of(int[]) has the same
trap
boxed[0] = 99;
System.out.println(right + " write-through to the array");
try { right.add(4); } catch (UnsupportedOperationException e) { System.out.println("fixed-size
list"); }
```

▶ **Output**

```
1 3
3
[1, 2, 3] 1
[99, 2, 3] write-through to the array
fixed-size list
```

> ⚠️ **Interview trap**
> `new ArrayList<>(Arrays.asList(prim))` doesn't compile to what you expect either: you get `ArrayList<int[]>`. Box the array first.

© TechTribers. For personal use only. Do not redistribute.

Series progress

529 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 13 starts at Q530.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.